

DaTra: Data Transformation Semantics as a Symmetric Monoidal Category on Atlas Presheaves

@qualiascript

ChatGPT

Abstract

This work formalizes the semantics of the Datra programming-language project in Lean. Data domains are organized into atlases: structured, eventually stable families of pages encoding how data is partitioned and related. Data transformations are modeled as presheaves on the resulting atlas categories, yielding a topos-theoretic semantics. Stable transformations are then assembled over atlas federations, whose horizontal sum extends by Day convolution to a symmetric monoidal category.

1 Dominions

Definition 1.1 (The Category of Dominions). The **Category of Dominions**, denoted $\mathbf{Dom}$, has as its objects sets whose cardinality is at most ω_0 , equivalently sets admitting an injection into ω_0 , and as its morphisms set-theoretic functions.

Definition 1.2 (The Domanial Embedding Functor). The **Domanial Embedding Functor**, denoted $\mathbf{Emb} : \mathbf{Dom} \rightarrow \mathbf{Set}$, sends each dominion to its underlying set.

2 Domanial Insertions

Definition 2.1 (The Category of Domanial Insertions). The **Category of Domanial Insertions**, denoted $\mathbf{DomIns}$, has dominions as objects and monomorphisms in $\mathbf{Dom}$ as morphisms.

Definition 2.2 (The Category of Codomanial Insertions). The **Category of Codomanial Insertions**, denoted $\mathbf{CoDomIns}$, is the opposite category $\mathbf{DomIns}^{\mathrm{op}}$.

3 Chains

Definition 3.1 (Chain). A **Chain** is a totally ordered thin category, indexed skeletally by an ordinal strictly smaller than $\omega_0^{\omega_0}$.

Definition 3.2 (The Spine). The **spine** is the chain with ω_0 objects.

4 Consolidations

Definition 4.1 (The Category of Consolidations). The **Category of Consolidations**, denoted $\mathbf{Con}$, has chains as objects and point-surjective functors between their thin categories as morphisms.

Definition 4.2 (The Category of Coconsolidations). The **Category of Coconsolidations**, denoted $\mathbf{CoCon}$, is the opposite category $\mathbf{Con}^{\mathrm{op}}$.

5 Transportation and Folios

Definition 5.1 (The Transportation Functor). The **Transportation Functor**, denoted $\text{Tra} : \text{Con} \rightarrow \text{Set}$, sends a chain to its object set and a consolidation to its underlying set-theoretic surjection.

Definition 5.2 (Folio). A **folio** is a functor $F : S \rightarrow \text{CoCon}$, where S is the spine, such that $F(0)$ is the singleton chain.

6 Paginations

Definition 6.1 (The Category of Paginations). The **Category of Paginations**, denoted Pag , has as objects pairs (H, E) where $H = \text{Tra} \circ F^{\text{op}}$ for a folio F , and E is the category of elements of H . For $W : \text{Pag}$, write W_H for the first inclusion and W_E for the second inclusion. A morphism $T : X \rightarrow Y$ in Pag is a functor $T : X_E \rightarrow Y_E$. Thus, for every morphism $f : x \rightarrow y$ of X_E , the following square commutes:

$$\begin{array}{ccc} x = (m, i) & \xrightarrow{f} & (n, X_H(f)(i)) \\ T \downarrow & & \downarrow T \\ (m', i') & \xrightarrow{T(f)} & (n', Y_H(T(f))(i')). \end{array}$$

7 Atlases

Definition 7.1 (The Category of Atlases). The **Category of Atlases**, denoted Atl , has as objects pairs (P, G) , where $P : \text{Pag}$ and $G : P_E \rightarrow \text{DomIns}$ is a functor, equivalently a presheaf $G : P_E^{\text{op}} \rightarrow \text{CoDomIns}$. For $P_H : C \rightarrow \text{Set}$, $m \in |C|$, and distinct elements $k, k' \in |P_H(m)|$, the pullback in G of the arrows induced by $(m \rightarrow 0)(k)$ and $(m \rightarrow 0)(k')$ is empty.

For every $n \in \mathbb{N}$, the n **th atlas page** of X is the set $X_H(n)$ on its full spine. An element $k \in X_H(n)$ is a **page cell**, written (n, k) , and the dominion carried by that cell is $X_G(n, k)$. Thus a page is the indexed collection of cells at one spine position.

There is an integer w such that, for every $w' > w$, $P_H(w' \rightarrow w) = \text{Id}$, and, for every $k \in |P_H(w)|$, the corresponding arrow $G((w' \rightarrow w)(k))$ is the identity. The least such integer is the **cardinality of the atlas**, denoted $|A|$.

For $X : \text{Atl}$, write X_P and X_G for the two components and put $X_H = (X_P)_H$ and $X_E = (X_P)_E$. A morphism $T : X \rightarrow Y$ is a pair (T_P, T_A) , where $T_P : X_P \rightarrow Y_P$ is a pagination morphism and $T_A : X_G \Rightarrow Y_G \circ T_E$ is a natural transformation. Thus, for every $f : x \rightarrow y$ in X_E , the following square commutes:

$$\begin{array}{ccc} X_G(x) & \xrightarrow{X_G(f)} & X_G(y) \\ (T_A)_x \downarrow & & \downarrow (T_A)_y \\ Y_G(T_E(x)) & \xrightarrow{Y_G(T_E(f))} & Y_G(T_E(y)). \end{array}$$

Finally, if $x = |X|$, $x' > x$, and $r \in |X_H(|X| - 1)|$, then $T_P(x', r) = T_P(x, r)$ and $(T_A)_{(x', r)} = (T_A)_{(x, r)}$.

Definition 7.2 (Extent of an Atlas). The **extent** of A is $\text{Ex}(A) = A_G(0, 0)$. Since objects of DomIns are dominions, $\text{Ex}(A) : \text{Dom}$ for every $A : \text{Atl}$.

Definition 7.3 (Territory of an Atlas). Let $M = A_H(|A| - 1)$. The **territory** is the indexed family $\text{Ter}(A) : M \rightarrow \text{Dom}$ given by $\text{Ter}(A)(k) = A_G(|A| - 1, k)$.

Definition 7.4 (*n*th Region of an Atlas). The *n*th **region** of A is $\text{Ter}(A)(n)$, with indexing beginning at 0.

8 Transposals and Traversals

Definition 8.1 (The Category of Atlas Transposals). The **Category of Atlas Transposals**, denoted AtlTrap , is the wide subcategory of Atl whose morphisms $F : X \rightarrow Y$ have point-injective object maps F_E .

Definition 8.2 (The Category of Atlas Traversals). The **Category of Atlas Traversals**, denoted AtlTrav , is the wide subcategory of AtlTrap whose morphisms satisfy the following conditions. For $F : X \rightarrow Y$, let $x = (m, i)$ and $y = (m', j)$, put $p = \min(m, m')$, write $F_E(x) = (n, i')$ and $F_E(y) = (n', j')$, and put $p' = \min(n, n')$. If

$$X_H(m \rightarrow p)(i) < X_H(m' \rightarrow p)(j),$$

then

$$Y_H(n \rightarrow p')(i') < Y_H(n' \rightarrow p')(j').$$

Furthermore, if $q \in |X|$ and $t \in X_G(q)$ is in the image of a final region—that is, there exist $k \in |\text{Ter}(X)|$ and $l \in \text{Ter}(X)(k)$ whose image under $X_G(|X| - 1 \rightarrow q)(k)$ is t —then this property is preserved by F .

Definition 8.3 (The Category of Stable Atlas Traversals). The **Category of Stable Atlas Traversals**, denoted StaAtlTrav , is the wide subcategory of AtlTrav whose morphisms $F : X \rightarrow Y$ satisfy $F_E(0, 0) = (0, 0)$.

9 Maps and Cartography

Definition 9.1 (The Category of Atlas Maps). The **Category of Atlas Maps**, denoted AtlMap , is the full subcategory of atlases for which every element of the extent is in the image of a final region.

Definition 9.2 (The Atlas Map Inclusion Functor). The **Atlas Map Inclusion Functor** $\text{AtlMapInc} : \text{AtlMap} \rightarrow \text{Atl}$ is the canonical inclusion.

Definition 9.3 (The Category of Atlas Traversal Maps). The **Category of Atlas Traversal Maps**, denoted AtlTravMap , is the full subcategory of AtlTrav whose objects are Atlas Maps.

Definition 9.4 (The Atlas Traversal Map Inclusion Functor). The canonical inclusion is denoted $\text{AtlTravMapInc} : \text{AtlTravMap} \rightarrow \text{AtlTrav}$.

Lemma 9.5 (Cartography Lemma). *The inclusion AtlTravMapInc has a right adjoint, denoted the **Charting Functor** $\text{Chr} : \text{AtlTrav} \rightarrow \text{AtlTravMap}$.*

Proof sketch. On an object $X : \text{AtlTrav}$, the functor Chr sends X to the universal arrow from AtlTravMapInc to X . Write $X' = \text{AtlTravMapInc}(\text{Chr}(X))$, together with $H : X' \rightarrow X$, with X' terminal among objects having this property. The solution is the DaTra map satisfying $\text{Ter}(X') \cong \text{Ter}(X)$. It is unique because AtlTrav preserves orders and terminal because H_E is faithful.

10 Coalitions

Definition 10.1 (The Coalizing Functor). The **Coalizing Functor**, denoted

$$\text{Coa} : \text{StaAtlTrav} \longrightarrow \text{DomIns},$$

is the extent of the charted atlas: $\text{Coa} = \text{Ex} \circ \text{Chr}$.

Definition 10.2 (The Coalition of an Atlas). The **coalition** of an atlas X is the extent of its chart, regarding X as an object of the wide category of stable traversals.

Definition 10.3 (The Domanial Inclusion Functor). The **Domanial Inclusion Functor** $\text{DomInc} : \text{DomIns} \rightarrow \text{StaAtlTrav}$ sends a dominion to the atlas of cardinality 1 whose coalition is that dominion. It is left adjoint to Coa : for $X : \text{DomIns}$ and $Y : \text{StaAtlTrav}$, naturally in X and Y ,

$$\text{Hom}_{\text{StaAtlTrav}}(\text{DomInc}(X), Y) \cong \text{Hom}_{\text{DomIns}}(X, \text{Coa}(Y)),$$

and $\text{Coa}(\text{DomInc}(X)) \cong X$.

11 Data Transformations

Definition 11.1 (The Category of Data Transformations). The **Category of Data Transformations**, also called the category of **Data Transformation Sets** or **DaTra Sets**, is the presheaf category

$$\text{DaTra} = [\text{Atl}^{\text{op}}, \text{Set}].$$

As a presheaf category, it is a topos.

Definition 11.2 (Navigation). A **navigation** of $D : \text{DaTra}$ is a monomorphism $\text{Nav} : \text{Yo}(A) \rightarrow D$ for some atlas A .

Definition 11.3 (Expedition). An **expedition** is a navigation represented by an Atlas Map.

Definition 11.4 (The Category of Data Transposals). The **Category of Data Transposals**, denoted DaTrap , is the presheaf category $[\text{AtlTrap}^{\text{op}}, \text{Set}]$.

Definition 11.5 (The Category of Data Traversals). The **Category of Data Traversals**, denoted DaTrav , is the presheaf category $[\text{AtlTrav}^{\text{op}}, \text{Set}]$.

Definition 11.6 (Data Transformation Maps). The **Category of Data Transformation Maps**, denoted DaTraMap , is the full subcategory of DaTra Sets all of whose navigations are expeditions.

12 Atlas Federations

Definition 12.1 (Atlas Merge). The **Atlas Merge** is the object operation

$$\text{AtlMerge} : \text{Ob}(\text{Atl}) \times \text{Ob}(\text{Atl}) \longrightarrow \text{Ob}(\text{Atl}).$$

For atlases X and Y , the extent of $\text{AtlMerge}(X, Y)$ is the disjoint union of their extents. Both atlases are evaluated on their full spines: page $n + 1$ of the merge is the tagged, componentwise combination of page n of X and page n of Y .

Definition 12.2 (Atlas Federation). An **Atlas Federation** is a countable tagged family of atlas objects. The category of Atlas Federations is denoted

$$\text{AtlFed}.$$

A morphism consists of a map of tags and, at each source tag, a stable atlas traversal to its selected target tag.

Definition 12.3 (The Empty Atlas). The **Empty Atlas** is $\text{AtII} = \text{DomInc}(\emptyset)$, also viewed as an Atlas Federation.

Definition 12.4 (The Atlas Horizontal Sum Bifunctor). The **Atlas Horizontal Sum Bifunctor**, denoted $\text{AtlHorSum} : \text{AtlFed} \times \text{AtlFed} \rightarrow \text{AtlFed}$, has object action

$$\text{AtlHorSum}(X, X') = \text{AtlMerge}(X, X').$$

Here AtlFed is the category of Atlas Federations; its empty federation represents AtII . Given stable traversals $F : X \rightarrow Y$ and $F' : X' \rightarrow Y'$, horizontal sum preserves the left and right tags and applies F and F' componentwise. Stability fixes the common origin, so this arrow action is again a stable atlas traversal.

Lemma 12.5 (Horizontal Lemma). *The Atlas Horizontal Sum Bifunctor AtlHorSum exists.*

Proof. Identity arrows act identically on every tag, composition holds componentwise, and every component arrow is required to lie in $\text{StaAtI}Trav$. The federation structure supplies the associator, unitors, and braider by reassociation, deletion of the empty tag family, and tag swapping, respectively.

Definition 12.6 (The Atlas Braider). The **Atlas Braider** $\text{AtlBrd}_{F, F'} : \text{AtlHorSum}(F, F') \rightarrow \text{AtlHorSum}(F', F)$ swaps the left and right component tags. Hence $\text{AtlBrd}_{F, F'} \circ \text{AtlBrd}_{F', F} = \text{Id}$.

Definition 12.7 (The Atlas Associator). The **Atlas Associator**, denoted

$$\text{AtlAsoc}_{F, F', F''} : \text{AtlHorSum}(\text{AtlHorSum}(F, F'), F'') \longrightarrow \text{AtlHorSum}(F, \text{AtlHorSum}(F', F'')),$$

flattens the nested federation and retags its three sides using the canonical reassociation $((x + y) + z) \leftrightarrow (x + (y + z))$. Its data component is the matching reassociation of the three extent summands.

Definition 12.8 (The Atlas Unitors). The **Atlas Left Unitor** $\text{Lu} : \text{AtlHorSum}(\text{AtII}, F) \rightarrow F$ and the **Atlas Right Unitor** $\text{Ru} : \text{AtlHorSum}(F, \text{AtII}) \rightarrow F$ remove the empty tag family.

13 Stable Data Transversals

Definition 13.1 (Stable Data Transversals). The **Category of Stable Data Transversals**, denoted StaDaTrav , is the presheaf category on Atlas Federations whose component arrows lie in $\text{StaAtI}Trav$.

Definition 13.2 (The Empty Map). The **Empty Map**, denoted I , is the Day unit represented by the empty Atlas Federation:

$$I = \text{Yo}(\text{AtII}).$$

Definition 13.3 (The Horizontal Sum Bifunctor). The **Horizontal Sum Bifunctor**, denoted $\text{HorSum} : \text{StaDaTrav} \times \text{StaDaTrav} \rightarrow \text{StaDaTrav}$, or in infix notation by $+_{<} : \text{StaDaTrav} \times \text{StaDaTrav} \rightarrow \text{StaDaTrav}$, is the Day convolution extension of stable atlas horizontal sum. Because the indexing morphisms are arrows of $\text{StaAtI}Trav$, its result and its arrow action land in StaDaTrav by construction.

Definition 13.4 (The Stable Data Transversal Braider). The **Stable Data Transversal Braider** $\text{Brd}_{F, F'} : F +_{<} F' \rightarrow F' +_{<} F$ is the Day convolution extension of AtlBrd .

Definition 13.5 (The Stable Data Transversal Associator). The **Stable Data Transversal Associator**

$$\text{Asoc}_{F, F', F''} : (F +_{<} F') +_{<} F'' \longrightarrow F +_{<} (F' +_{<} F'')$$

is the Day convolution extension of AtlAsoc .

Definition 13.6 (The Stable Data Transversal Unitors). The **Stable Data Transversal Left Unitor** $\text{Lu} : I +_{<} F \rightarrow F$ and the **Stable Data Transversal Right Unitor** $\text{Ru} : F +_{<} I \rightarrow F$ are the Day convolution extensions of AtlLu and AtlRu .

Definition 13.7 (Stable Data Transversals Monoidal Category). The **Stable Data Transversals Monoidal Category**, denoted StaDaTravMon , is

$$\text{StaDaTravMon} = (\text{StaDaTrav}, +_{<}, I).$$

Its tensor is Day convolution on the Atlas Federation. The braider, associator, and unitors are induced by retagging that form, so all coherence maps remain within stable data transversals.

A Lean Formalization

The following is an ASCII rendering of the complete Lean source corresponding to the mathematical development above. It replaces Lean’s conventional Unicode surface notation solely for portable typesetting; the checked source file remains `datra.lean`.

```
1  import Mathlib
2
3  namespace Datra
4
5  open CategoryTheory CategoryTheory.Functor CategoryTheory.Limits Opposite
6  open scoped Ordinal
7
8  universe u
9
10 noncomputable section
11
12 /-- A dominion is a set equipped with a certificate of countability. -/
13 structure Dominion where
14   Carrier : Type
15   rank : Function.Embedding Carrier Nat
16
17 abbrev Dom := Dominion
18
19 instance : CoeSort Dominion Type where coe X := X.Carrier
20
21 instance : Category Dominion where
22   Hom X Y := X -> Y
23   id _ := id
24   comp f g := g o f
25   id_comp _ := rfl
26   comp_id _ := rfl
27   assoc _ _ _ := rfl
28
29 def Emb : Dom ==> Type where
30   obj X := X
31   map f := f
32
33 structure DomIns where
34   toDom : Dom
35
36 instance : CoeSort DomIns Type where coe X := X.toDom.Carrier
37
38 instance : Category DomIns where
39   Hom X Y := Function.Embedding X Y
40   id X := Function.Embedding.refl X
41   comp f g := f.trans g
42   id_comp f := by ext; rfl
43   comp_id f := by ext; rfl
44   assoc f g h := by ext; rfl
45
46 instance {X Y : DomIns} : CoeFun (X --> Y) (fun _ => X -> Y) where
47   coe f := f.toFun
48
49 @[ext]
50 theorem DomIns.hom_ext {X Y : DomIns} (f g : X --> Y)
51   (h : forall x, f x = g x) : f = g := by
52   exact Function.Embedding.ext h
53
54 instance {X Y : DomIns} (f : X --> Y) : Mono f where
55   right_cancellation g h w := by
56     apply DomIns.hom_ext
57     intro x
58     apply f.injective
59     exact congrFun (congrArg Function.Embedding.toFun w) x
60
61 def DomIns.forget : DomIns ==> Dom where
62   obj X := X.toDom
63   map f := f.toFun
64
65 abbrev CoDomIns := Opposite DomIns
66
```

```

67 /-- A chain carries its skeletal well-ordered object type and a proof that
68 its order type lies below  $\omega_0 \sim \omega_0$ . Retaining the carrier explicitly makes
69 ordinal sums of chains definitionally usable. -/
70 structure Chain where
71   Obj : Type
72   linearOrder : LinearOrder Obj
73   wellFoundedLT : WellFoundedLT Obj
74   orderType_lt : Ordinal.type (fun x y : Obj => x < y) <
75     Ordinal.omega0 ~ Ordinal.omega0
76
77 attribute [instance] Chain.linearOrder Chain.wellFoundedLT
78
79 def Chain.sum (X Y : Chain) : Chain where
80   Obj := Lex (Sum X.Obj Y.Obj)
81   linearOrder := inferInstance
82   wellFoundedLT := <|by
83     change WellFounded (Sum.Lex (fun x y : X.Obj => x < y)
84       (fun x y : Y.Obj => x < y))
85     exact Sum.lex_wf wellFounded_lt wellFounded_lt|>
86   orderType_lt := by
87     change Ordinal.type (Sum.Lex (fun x y : X.Obj => x < y)
88       (fun x y : Y.Obj => x < y)) < Ordinal.omega0 ~ Ordinal.omega0
89   rw [Ordinal.type_sum_lex]
90   exact Ordinal.principal_add_omega0_opow Ordinal.omega0
91     X.orderType_lt Y.orderType_lt
92
93 /-- The common page-indexing chain. -/
94 def Spine : Chain where
95   Obj := Nat
96   linearOrder := inferInstance
97   wellFoundedLT := inferInstance
98   orderType_lt := by
99     have hpow : Ordinal.omega0 ~ (1 : Ordinal) <
100       Ordinal.omega0 ~ Ordinal.omega0 :=
101       (Ordinal.opow_lt_opow_iff_right Ordinal.one_lt_omega0).2
102     Ordinal.one_lt_omega0
103     change typeLT Nat < Ordinal.omega0 ~ Ordinal.omega0
104     rw [Ordinal.type_nat_lt]
105     simpa only [Ordinal.opow_one] using hpow
106
107 abbrev Con := Chain
108
109 /-- A functor between ordinal chains is equivalently a monotone object map. -/
110 structure ConHom (X Y : Con) where
111   toFun : X.Obj -> Y.Obj
112   monotone : Monotone toFun
113   point_surjective : Function.Surjective toFun
114
115 def ConHom.sum {X X' Y Y' : Con} (f : ConHom X Y) (g : ConHom X' Y') :
116   ConHom (Chain.sum X X') (Chain.sum Y Y') where
117   toFun z := toLex (Sum.map f.toFun g.toFun (ofLex z))
118   monotone := by
119     intro a b h
120     change Sum.Lex (fun x y : X.Obj => x <= y) (fun x y : X'.Obj => x <= y)
121       (ofLex a) (ofLex b) at h
122     change Sum.Lex (fun x y : Y.Obj => x <= y) (fun x y : Y'.Obj => x <= y)
123       (Sum.map f.toFun g.toFun (ofLex a))
124       (Sum.map f.toFun g.toFun (ofLex b))
125     generalize ha : ofLex a = sa at h | -
126     generalize hb : ofLex b = sb at h | -
127     rcases sa with x | x <;> rcases sb with y | y
128     .
129     simp only [Sum.map]
130     exact Sum.Lex.inl (f.monotone (Sum.lex_inl_inl.mp h))
131     . simp only [Sum.map]
132     exact Sum.Lex.sep _ _
133     . exact (Sum.lex_inr_inl h).elim
134     .
135     simp only [Sum.map]
136     exact Sum.Lex.inr (g.monotone (Sum.lex_inr_inr.mp h))
137   point_surjective := by
138     intro z
139     rcases hz : ofLex z with y | y

```



```

140   . obtain <|x, rfl|> := f.point_surjective y
141   refine <|toLex (Sum.inl x), ?_|>
142   apply toLex.injective
143   simpa using hz.symm
144   . obtain <|x, rfl|> := g.point_surjective y
145   refine <|toLex (Sum.inr x), ?_|>
146   apply toLex.injective
147   simpa using hz.symm
148
149 instance {X Y : Con} : CoeFun (ConHom X Y) (fun _ => X.Obj -> Y.Obj) where
150   coe f := f.toFun
151
152 @[ext]
153 theorem ConHom.ext {X Y : Con} (f g : ConHom X Y)
154   (h : forall x, f x = g x) : f = g := by
155   cases f
156   cases g
157   simp_all only [mk.injEq]
158   exact funext h
159
160 instance : Category.{0} Con where
161   Hom := ConHom
162   id X := <|id, monotone_id, Function.surjective_id|>
163   comp f g := <|g o f, g.monotone.comp f.monotone,
164     g.point_surjective.comp f.point_surjective|>
165   id_comp f := by ext; rfl
166   comp_id f := by ext; rfl
167   assoc f g h := by ext; rfl
168
169 instance {X Y : Con} : CoeFun (X --> Y) (fun _ => X.Obj -> Y.Obj) where
170   coe f := f.toFun
171
172 def Con.asFunctor {X Y : Con} (f : X --> Y) : X.Obj ==> Y.Obj where
173   obj := f
174   map g := homOfLE (f.monotone (leOfHom g))
175
176 abbrev CoCon := Opposite Con
177
178 def Tra : Con ==> Type where
179   obj X := X.Obj
180   map f := f.toFun
181
182 /-- A folio is stored by the least finite presentation of its eventually
183 constant spine functor. `length` is its cardinality: page `length - 1` is
184 the final genuine page and every later spine page repeats it. -/
185 structure Folio where
186   length : Nat
187   positive : 0 < length
188   core : Functor.{0, 0} (Fin length) CoCon
189   originEquiv : (core.obj <|0, positive|>).unop.Obj Equiv Unit
190
191 def Folio.paddedIndex (W : Folio) (n : Nat) : Fin W.length :=
192   <|min n (W.length - 1), lt_of_le_of_lt (min_le_right _ _)
193     (Nat.sub_lt W.positive (by omega))|>
194
195 theorem Folio.paddedIndex_mono (W : Folio) :
196   Monotone W.paddedIndex := by
197   intro a b hab
198   exact min_le_min hab le_rfl
199
200 @[simp]
201 theorem Folio.paddedIndex_fin (W : Folio) (m : Fin W.length) :
202   W.paddedIndex m.1 = m := by
203   apply Fin.ext
204   simp only [Folio.paddedIndex]
205   exact Nat.min_eq_left (by omega)
206
207 def Folio.spineBase (W : Folio) : Nat ==> Fin W.length where
208   obj n := W.paddedIndex n
209   map f := homOfLE (W.paddedIndex_mono (leOfHom f))
210   map_id _ := Subsingleton.elim _ _
211   map_comp _ _ := Subsingleton.elim _ _
212

```

```

213 /-- The actual functor on the spine denoted by the finite presentation. -/
214 def Folio.F (W : Folio) : Nat ==> CoCon := W.spineBase then W.core
215
216 /-- The page functor on the opposite spine. -/
217 def Folio.spineH (W : Folio) : Natop ==> Type := W.F.leftOp then Tra
218
219 def Folio.H (W : Folio) : (Fin W.length)op ==> Type := W.core.leftOp then Tra
220
221 def Folio.E (W : Folio) : Type 0 := W.H.Elements
222
223 instance (W : Folio) : Category.{0} W.E := categoryOfElements W.H
224
225 instance (W : Folio) (m : (Fin W.length)op) : LinearOrder (W.H.obj m) :=
226   (W.core.obj m.unop).unop.linearOrder
227
228 instance (W : Folio) (x y : W.E) : Subsingleton (x --> y) where
229   allEq f g := CategoryOfElements.ext W.H f g (Subsingleton.elim _ _)
230
231 def Folio.originIndex (W : Folio) : Fin W.length := <|0, W.positive|>
232 def Folio.originBase (W : Folio) : (Fin W.length)op := op W.originIndex
233 def Folio.originValue (W : Folio) : W.H.obj W.originBase :=
234   W.originEquiv.symm ()
235 def Folio.originElement (W : Folio) : W.E := <|W.originBase, W.originValue|>
236
237 def Folio.lastIndex (W : Folio) : Fin W.length :=
238   <|W.length - 1, Nat.sub_lt W.positive (by omega)|>
239 def Folio.lastBase (W : Folio) : (Fin W.length)op := op W.lastIndex
240
241 theorem Folio.origin_unique (W : Folio) (x : W.H.obj W.originBase) :
242   x = W.originValue := by
243   apply W.originEquiv.injective
244   exact Subsingleton.elim _ _
245
246 def Folio.baseToOrigin (W : Folio) (m : (Fin W.length)op) :
247   m --> W.originBase := by
248   letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
249   exact (homOfLE (show W.originIndex <= m.unop by
250     change (0 : Fin W.length) <= m.unop
251     exact bot_le)).op
252
253 def Folio.toOrigin (W : Folio) (x : W.E) : x --> W.originElement :=
254   CategoryOfElements.homMk x W.originElement (W.baseToOrigin x.1)
255   (W.origin_unique _)
256
257 /-- The tall presentation is kept internal. A folio's listed cardinality is
258 only a finite presentation of its genuinely `Nat`-indexed spine. -/
259
260 def Folio.TallE (W : Folio) : Type := W.spineH.Elements
261
262 instance (W : Folio) : Category W.TallE := categoryOfElements W.spineH
263
264 instance (W : Folio) (x y : W.TallE) : Subsingleton (x --> y) where
265   allEq f g := CategoryOfElements.ext W.spineH f g (Subsingleton.elim _ _)
266
267 /-- Collapse a tall occurrence to the coherent finite representative used for
268 storage. This is an implementation map, not the page space of the atlas. -/
269 def Folio.collapseElements (W : Folio) : W.TallE ==> W.E where
270   obj x := <|op (W.paddedIndex x.1.unop), x.2|>
271   map {x y} f := CategoryOfElements.homMk _ _
272     (W.spineBase.map f.val.unop).op f.property
273   map_id _ := by
274     apply CategoryOfElements.ext
275     apply Subsingleton.elim
276   map_comp _ _ := by
277     apply CategoryOfElements.ext
278     apply Subsingleton.elim
279
280 /-- Include a stored cell at its actual page into the full spine. -/
281 def Folio.includeElements (W : Folio) : W.E ==> W.TallE where
282   obj x := <|op x.1.unop.1,
283     W.H.map (eqToHom
284       (congrArg op (W.paddedIndex_fin x.1.unop).symm)) x.2|>
285   map {x y} f := by

```

```

286   rcases x with <|<|m|>, k|>
287   rcases y with <|<|n|>, l|>
288   have hnm : n.1 <= m.1 := leOfHom f.val.unop
289   refine CategoryOfElements.homMk _ _ (homOfLE hnm).op ?_
290   simp only [Folio.spineH, Folio.F, Folio.spineBase]
291   change W.H.map _ (W.H.map _ k) = W.H.map _ l
292   rw [<- FunctorToTypes.map_comp_apply,
293       show _ >> _ = f.val >> _ from Subsingleton.elim _ _]
294   rw [FunctorToTypes.map_comp_apply, f.property]
295   map_id _ := by
296     apply CategoryOfElements.ext
297     apply Subsingleton.elim
298   map_comp _ _ := by
299     apply CategoryOfElements.ext
300     apply Subsingleton.elim
301
302 @[simp]
303 theorem Folio.collapse_include (W : Folio) :
304   W.includeElements then W.collapseElements = Unit W.E := by
305   exact CategoryTheory.Functor.ext
306   (F := W.includeElements then W.collapseElements) (G := Functor.id W.E)
307   (fun x => by
308     rcases x with <|<|m|>, k|>
309     refine Functor.Elements.ext _ _ (congrArg op (W.paddedIndex_fin m)) ?_
310     simp [Folio.includeElements, Folio.collapseElements])
311
312 @[simp]
313 theorem Folio.collapse_include_obj (W : Folio) (x : W.E) :
314   W.collapseElements.obj (W.includeElements.obj x) = x := by
315   exact Functor.congr_obj W.collapse_include x
316
317 /-- The finite presentation is a retract of the full spine, but not conversely:
318 collapsing the first repeated page and including it again changes its page
319 index. This records the precise obstruction to treating the two atlas bases
320 as interchangeable through the evident inclusion/collapse comparison. -/
321 theorem Folio.include_collapse_ne_id (W : Folio) :
322   W.collapseElements then W.includeElements != Functor.id W.TallE := by
323   intro h
324   letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
325   let q : (W.core.obj W.lastIndex).unop -->
326     (W.core.obj W.originIndex).unop :=
327     (W.core.map (homOfLE (Fin.zero_le W.lastIndex))).unop
328   let k : (W.core.obj W.lastIndex).unop.Obj :=
329     Classical.choose (q.point_surjective W.originValue)
330   let x : W.TallE := by
331     refine <|op W.length, ?_|>
332     change (W.core.obj (W.paddedIndex W.length)).unop.Obj
333     have hp : W.paddedIndex W.length = W.lastIndex := by
334       apply Fin.ext
335       simp [Folio.paddedIndex, Folio.lastIndex]
336     exact hp.symm transport k
337   have hx := congrArg
338     (fun F : CategoryTheory.Functor W.TallE W.TallE => (F.obj x).1.unop) h
339   have hbad : W.length - 1 = W.length := by
340     simp [x, Folio.collapseElements, Folio.includeElements,
341         Folio.paddedIndex] using hx
342   exact (ne_of_lt (Nat.sub_lt W.positive (by omega))) hbad
343
344 structure Pag where
345   folio : Folio
346
347 def Pag.H (W : Pag) : (Fin W.folio.length).op ==> Type 0 := W.folio.H
348 def Pag.E (W : Pag) : Type 0 := W.folio.E
349
350 instance (W : Pag) : Category.{0} W.E := categoryOfElements W.H
351
352 instance : Category.{0} Pag where
353   Hom X Y := X.E ==> Y.E
354   id X := Unit X.E
355   comp F G := F then G
356   id_comp _ := rfl
357   comp_id _ := rfl
358   assoc _ _ _ := rfl

```

```

359
360 /-- Internally, an atlas is stored by its least finite presentation. The
361 `Folio.F` construction repeats its final page along the rest of the spine, so
362 the stabilization and morphism-tail clauses in the extracted definition are
363 enforced by construction. -/
364
365 def Pag.cell (P : Pag) (m : (Fin P.folio.length)op) (k : P.H.obj m) : P.E := <|m, k|>
366
367 def Pag.cellToOrigin (P : Pag) (m : (Fin P.folio.length)op)
368   (k : P.H.obj m) : P.cell m k --> P.folio.originElement :=
369   P.folio.toOrigin (P.cell m k)
370
371 def IsPagewiseDisjoint (P : Pag) (G : P.E ==> DomIns) : Prop :=
372   forall (m : (Fin P.folio.length)op) (i j : P.H.obj m), i != j ->
373     forall (x : G.obj (P.cell m i)) (y : G.obj (P.cell m j)),
374       G.map (P.cellToOrigin m i) x != G.map (P.cellToOrigin m j) y
375
376 structure Atl where
377   P : Pag
378   G : P.E ==> DomIns
379   disjoint : IsPagewiseDisjoint P G
380
381 def Atl.H (X : Atl) : (Fin X.P.folio.length)op ==> Type 0 := X.P.H
382 def Atl.E (X : Atl) : Type 0 := X.P.E
383
384 /-- The `n`th page of an atlas on its genuine infinite spine. -/
385 def Atl.page (X : Atl) (n : Nat) : Type := X.P.folio.spineH.obj (op n)
386
387 /-- A cell of the `n`th page, retaining its position on the infinite spine. -/
388 def Atl.pageCell (X : Atl) (n : Nat) (k : X.page n) : X.P.folio.TallE :=
389   <|op n, k|>
390
391 instance (X : Atl) : Category.{0} X.E := categoryOfElements X.H
392
393 structure AtlHom (X Y : Atl) where
394   P : X.E ==> Y.E
395   A : X.G --> P then Y.G
396
397 def AtlHom.identity (X : Atl) : AtlHom X X where
398   P := Unit X.E
399   A := Id X.G
400
401 def AtlHom.comp {X Y Z : Atl} (f : AtlHom X Y) (g : AtlHom Y Z) : AtlHom X Z where
402   P := f.P then g.P
403   A := f.A >> whiskerLeft f.P g.A
404
405 @[ext]
406 theorem AtlHom.ext {X Y : Atl} (f g : AtlHom X Y)
407   (hP : f.P = g.P) (hA : HEq f.A g.A) : f = g := by
408   cases f
409   cases g
410   simp_all
411
412 /-- Heterogeneous extensionality for natural transformations whose target
413 functors have been identified. This keeps dependent transports localized. -/
414 theorem NatTrans.hext_right {C D : Type*} [Category C] [Category D]
415   {F G H : C ==> D} (alpha : F --> G) (beta : F --> H) (h : G = H)
416   (happ : forall X, HEq (alpha.app X) (beta.app X)) : HEq alpha beta := by
417   cases h
418   apply heq_of_eq
419   ext X
420   exact eq_of_heq (happ X)
421
422 /-- Transporting the codomain of an embedding along an equality of objects is
423 heterogeneously equal to the original embedding. This is the small coherence
424 fact needed whenever stability identifies the image of an origin with the
425 target origin. -/
426 theorem embedding_comp_map_eqToHom_heq {C : Type u} [Category C]
427   (G : CategoryTheory.Functor C DomIns) {a b : C} (h : a = b) {X : DomIns}
428   (e : X --> G.obj a) : HEq (e >> G.map (eqToHom h)) e := by
429   cases h
430   simp
431

```

```

432 instance : Category.{0} Atl where
433   Hom := AtlHom
434   id := AtlHom.identity
435   comp := AtlHom.comp
436   id_comp f := by
437     apply AtlHom.ext
438     . rfl
439     . apply heq_of_eq
440     ext x
441     simp [AtlHom.comp, AtlHom.identity]
442   comp_id f := by
443     apply AtlHom.ext
444     . rfl
445     . apply heq_of_eq
446     ext x
447     simp [AtlHom.comp, AtlHom.identity]
448   assoc f g h := by
449     apply AtlHom.ext
450     . rfl
451     . apply heq_of_eq
452     ext x
453     simp [AtlHom.comp]
454
455 def Folio.commonBase (W : Folio) (m n : (Fin W.length)op) :
456   (Fin W.length)op := op (min m.unop n.unop)
457
458 def Folio.toCommonLeft (W : Folio) (m n : (Fin W.length)op) :
459   m --> W.commonBase m n := (homOfLE (min_le_left _ _)).op
460
461 def Folio.toCommonRight (W : Folio) (m n : (Fin W.length)op) :
462   n --> W.commonBase m n := (homOfLE (min_le_right _ _)).op
463
464 def storedElementLT (X : Atl) (x y : X.E) : Prop :=
465   let m := X.P.folio.commonBase x.1 y.1
466   let l : LinearOrder (X.H.obj m) :=
467     (X.P.folio.core.obj m.unop).unop.linearOrder
468   X.H.map (X.P.folio.toCommonLeft x.1 y.1) x.2 <
469   X.H.map (X.P.folio.toCommonRight x.1 y.1) y.2
470
471 def storedExtent (A : Atl) : DomIns := A.G.obj A.P.folio.originElement
472
473 def StoredTerritoryIndex (A : Atl) : Type := A.H.obj A.P.folio.lastBase
474
475 def storedTerritory (A : Atl) (k : StoredTerritoryIndex A) : DomIns :=
476   A.G.obj (A.P.cell A.P.folio.lastBase k)
477
478 def storedOriginImage (X : Atl) (x : X.E) (t : X.G.obj x) : storedExtent X :=
479   X.G.map (X.P.folio.toOrigin x) t
480
481 def storedCovered (X : Atl) (x : X.E) (t : X.G.obj x) : Prop :=
482   exists (k : X.H.obj X.P.folio.lastBase)
483     (l : X.G.obj (X.P.cell X.P.folio.lastBase k)),
484     storedOriginImage X x t =
485     storedOriginImage X (X.P.cell X.P.folio.lastBase k) l
486
487 #!/ `TallAtlas` is the internal, genuinely infinite-spine presentation. An
488 `Atl` supplies the coherent page data by pulling its finite storage back along
489 `collapseElements`. This distinction deliberately stays out of the extracted
490 mathematical definitions. -/
491
492 structure TallAtlas where
493   H : Natop ==> Type
494   originValue : H.obj (op 0)
495   originUnique : forall x : H.obj (op 0), x = originValue
496   G : H.Elements ==> DomIns
497   cellLT : H.Elements -> H.Elements -> Prop
498   coveredExtent : Set (G.obj <|op 0, originValue|>)
499
500 def TallAtlas.E (X : TallAtlas) : Type := X.H.Elements
501
502 instance (X : TallAtlas) : Category X.E := categoryOfElements X.H
503
504 instance (X : TallAtlas) (x y : X.E) : Subsingleton (x --> y) where

```

```

505 allEq f g := CategoryOfElements.ext X.H f g (Subsingleton.elim _ _)
506
507 def Atl.tallOriginValue (X : Atl) : X.P.folio.spineH.obj (op 0) := by
508   simp [Folio.spineH, Folio.F, Folio.spineBase, Folio.paddedIndex,
509         Folio.originIndex] using X.P.folio.originValue
510
511 def Atl.tall (X : Atl) : TallAtlas where
512   H := X.P.folio.spineH
513   originValue := X.tallOriginValue
514   originUnique x := by
515     let e : X.P.folio.spineH.obj (op 0) Equiv Unit := by
516       simp [Folio.spineH, Folio.F, Folio.spineBase, Folio.paddedIndex,
517             Folio.originIndex] using X.P.folio.originEquiv
518     apply e.injective
519     exact Subsingleton.elim _ _
520   G := X.P.folio.collapseElements then X.G
521   cellLT x y := storedElementLT X
522     (X.P.folio.collapseElements.obj x)
523     (X.P.folio.collapseElements.obj y)
524   coveredExtent t := storedCovered X
525     (X.P.folio.collapseElements.obj
526      <|op 0, X.tallOriginValue|>) t
527
528 def TallAtlas.originElement (X : TallAtlas) : X.E :=
529   <|op 0, X.originValue|>
530
531 def TallAtlas.toOrigin (X : TallAtlas) (x : X.E) : x --> X.originElement :=
532   CategoryOfElements.homMk x X.originElement
533   (homOfLE (Nat.zero_le x.1.unop)).op (X.originUnique _)
534
535 def TallAtlas.covered (X : TallAtlas) (x : X.E) (t : X.G.obj x) : Prop :=
536   X.coveredExtent (X.G.map (X.toOrigin x) t)
537
538 def TallAtlas.extent (X : TallAtlas) : DomIns := X.G.obj X.originElement
539
540 structure TallAtlasHom (X Y : TallAtlas) where
541   P : X.E ==> Y.E
542   A : X.G --> P then Y.G
543
544 def TallAtlasHom.identity (X : TallAtlas) : TallAtlasHom X X where
545   P := Unit X.E
546   A := Id X.G
547
548 def TallAtlasHom.comp {X Y Z : TallAtlas}
549   (f : TallAtlasHom X Y) (g : TallAtlasHom Y Z) : TallAtlasHom X Z where
550   P := f.P then g.P
551   A := f.A >> whiskerLeft f.P g.A
552
553 @[ext]
554 theorem TallAtlasHom.ext {X Y : TallAtlas} (f g : TallAtlasHom X Y)
555   (hP : f.P = g.P) (hA : HEq f.A g.A) : f = g := by
556   cases f
557   cases g
558   simp_all
559
560 instance : Category TallAtlas where
561   Hom := TallAtlasHom
562   id := TallAtlasHom.identity
563   comp := TallAtlasHom.comp
564   id_comp f := by
565     apply TallAtlasHom.ext
566     . rfl
567     . apply heq_of_eq
568     ext x
569     simp [TallAtlasHom.comp, TallAtlasHom.identity]
570   comp_id f := by
571     apply TallAtlasHom.ext
572     . rfl
573     . apply heq_of_eq
574     ext x
575     simp [TallAtlasHom.comp, TallAtlasHom.identity]
576   assoc f g h := by
577     apply TallAtlasHom.ext

```

```

578 . rfl
579 . apply heq_of_eq
580 . ext x
581 . simp [TallAtlasHom.comp]
582
583 /-- An atlas arrow's action on the infinite presentation is derived from its
584 finite pagination and data transformation. The full spine is first collapsed
585 to the coherent stored representative, transformed by `P` and `A`, and then
586 included at the resulting stored page. Thus no independent spine action is
587 part of an atlas morphism. -/
588
589 def AtlHom.tallP {X Y : Atl} (f : X --> Y) :
590   X.tall.E ==> Y.tall.E :=
591   X.P.folio.collapseElements then f.P then Y.P.folio.includeElements
592
593 def AtlHom.tallA {X Y : Atl} (f : X --> Y) :
594   X.tall.G --> f.tallP then Y.tall.G where
595   app x := f.A.app (X.P.folio.collapseElements.obj x) >>
596     Y.G.map (eqToHom (Y.P.folio.collapse_include_obj
597       (f.P.obj (X.P.folio.collapseElements.obj x))).symm)
598   naturality := by
599     intro x y q
600     simp only [AtlHom.tallP, Atl.tall, Functor.comp_obj, Functor.comp_map]
601     rw [← Category.assoc, f.A.naturality, Category.assoc]
602     simp only [Functor.comp_map]
603     have he :
604       Y.G.map (f.P.map (X.P.folio.collapseElements.map q)) >>
605         Y.G.map (eqToHom (Y.P.folio.collapse_include_obj _).symm) =
606       Y.G.map (eqToHom (Y.P.folio.collapse_include_obj _).symm) >>
607         Y.G.map (Y.P.folio.collapseElements.map
608           (Y.P.folio.includeElements.map
609             (f.P.map (X.P.folio.collapseElements.map q)))) := by
610       rw [← Y.G.map_comp, ← Y.G.map_comp]
611       congr 1
612     rw [he]
613     simp only [Category.assoc]
614
615 def AtlHom.tall {X Y : Atl} (f : X --> Y) : X.tall --> Y.tall where
616   P := f.tallP
617   A := f.tallA
618
619 /-- The coherent identity of an infinite presentation. It identifies every
620 repeated occurrence with the stored representative. -/
621 def Atl.coherence (X : Atl) : X.tall --> X.tall :=
622   (AtlHom.identity X).tall
623
624 /-- Deriving the spine action commutes strictly with composition. -/
625 theorem AtlHom.tall_comp {X Y Z : Atl} (f : X --> Y) (g : Y --> Z) :
626   (f >> g).tall = f.tall >> g.tall := by
627   have hP : (f >> g).tall.P = (f.tall >> g.tall).P := by
628     exact CategoryTheory.Functor.ext
629     (F := (f >> g).tall.P) (G := (f.tall >> g.tall).P)
630     (fun x => by
631       change Z.P.folio.includeElements.obj
632         (g.P.obj (f.P.obj (X.P.folio.collapseElements.obj x))) =
633       Z.P.folio.includeElements.obj
634         (g.P.obj (Y.P.folio.collapseElements.obj
635           (Y.P.folio.includeElements.obj
636             (f.P.obj (X.P.folio.collapseElements.obj x)))))
637       rw [Y.P.folio.collapse_include_obj])
638   apply TallAtlasHom.ext _ _ hP
639   apply NatTrans.hex_right _ _
640   (congrArg (fun P => P then Z.tall.G) hP)
641   intro x
642   let x_0 := X.P.folio.collapseElements.obj x
643   let y_0 := f.P.obj x_0
644   let eY : y_0 = Y.P.folio.collapseElements.obj
645     (Y.P.folio.includeElements.obj y_0) :=
646     (Y.P.folio.collapse_include_obj y_0).symm
647   let z_0 := g.P.obj y_0
648   let eZ : z_0 = Z.P.folio.collapseElements.obj
649     (Z.P.folio.includeElements.obj z_0) :=
650     (Z.P.folio.collapse_include_obj z_0).symm

```

```

651 let y_1 := Y.P.folio.collapseElements.obj
652   (Y.P.folio.includeElements.obj y_0)
653 let z_1 := g.P.obj y_1
654 let eZ_1 : z_1 = Z.P.folio.collapseElements.obj
655   (Z.P.folio.includeElements.obj z_1) :=
656   (Z.P.folio.collapse_include_obj z_1).symm
657 let eR : z_0 = Z.P.folio.collapseElements.obj
658   (Z.P.folio.includeElements.obj z_1) :=
659   (congrArg g.P.obj eY).trans eZ_1
660 change HEq
661   ((AtlHom.comp f g).tallA.app x)
662   ((TallAtlasHom.comp f.tall g.tall).A.app x)
663 dsimp [AtlHom.tall, AtlHom.tallA, AtlHom.tallP, AtlHom.comp,
664   TallAtlasHom.comp]
665 rw [Category.assoc (f.A.app x_0) (Y.G.map (eqToHom eY))
666   (g.A.app y_1 >> Z.G.map (eqToHom eZ_1))]
667 rw [g.A.naturality_assoc (eqToHom eY)]
668 change HEq
669   ((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (eqToHom eZ))
670   (((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (g.P.map (eqToHom eY))) >>
671     Z.G.map (eqToHom eZ_1))
672 have hrArrow : g.P.map (eqToHom eY) >> eqToHom eZ_1 = eqToHom eR :=
673   CategoryOfElements.ext Z.H _ _ (Subsingleton.elim _ _)
674 have hright : HEq
675   (((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (g.P.map (eqToHom eY))) >>
676     Z.G.map (eqToHom eZ_1))
677   (f.A.app x_0 >> g.A.app y_0) := by
678   rw [Category.assoc, <- Z.G.map_comp, hrArrow]
679   exact embedding_comp_map_eqToHom_heq Z.G eR _
680   exact (embedding_comp_map_eqToHom_heq Z.G eZ
681     (f.A.app x_0 >> g.A.app y_0)).trans hright.symm
682
683 theorem Atl.coherence_idempotent (X : Atl) :
684   X.coherence >> X.coherence = X.coherence := by
685   have h := AtlHom.tall_comp (Id X) (Id X)
686   simpa [Atl.coherence] using h.symm
687
688 theorem Atl.coherence_ne_identity (X : Atl) :
689   X.coherence != Id X.tall := by
690   intro h
691   apply X.P.folio.include_collapse_ne_id
692   simpa [Atl.coherence, AtlHom.tall, AtlHom.tallP, AtlHom.identity,
693     TallAtlasHom.identity] using congrArg TallAtlasHom.P h
694
695 theorem AtlHom.coherence_left {X Y : Atl} (f : X --> Y) :
696   X.coherence >> f.tall = f.tall := by
697   have h := AtlHom.tall_comp (Id X) f
698   simpa [Atl.coherence] using h.symm
699
700 theorem AtlHom.coherence_right {X Y : Atl} (f : X --> Y) :
701   f.tall >> Y.coherence = f.tall := by
702   have h := AtlHom.tall_comp f (Id Y)
703   simpa [Atl.coherence] using h.symm
704
705 def cardinality (A : Atl) : Nat := A.P.folio.length
706
707 def extent (A : Atl) : DomIns := storedExtent A
708
709 def TerritoryIndex (A : Atl) : Type := StoredTerritoryIndex A
710
711 def territory (A : Atl) (k : TerritoryIndex A) : DomIns :=
712   storedTerritory A k
713
714 def region (A : Atl) (n : TerritoryIndex A) : DomIns := territory A n
715
716 def IsTransposal : MorphismProperty Atl :=
717   fun _ _ F => Function.Injective F.P.obj
718
719 instance : IsTransposal.IsMultiplicative where
720   id_mem _ := Function.injective_id
721   comp_mem _ _ hf hg := hg.comp hf
722
723 abbrev AtlTrap := WideSubcategory IsTransposal

```



```

724
725 def AtlTrapInc : AtlTrap ==> Atl := wideSubcategoryInclusion IsTransposal
726
727 def elementLT (X : Atl) (x y : X.E) : Prop :=
728   let m := X.P.folio.commonBase x.1 y.1
729   let l : LinearOrder (X.H.obj m) :=
730     (X.P.folio.core.obj m.unop).unop.linearOrder
731   X.H.map (X.P.folio.toCommonLeft x.1 y.1) x.2 <
732   X.H.map (X.P.folio.toCommonRight x.1 y.1) y.2
733
734 /-- The image of a datum in the extent. -/
735 def originImage (X : Atl) (x : X.E) (t : X.G.obj x) : extent X :=
736   storedOriginImage X x t
737
738 /-- A datum is covered when its image in the extent comes from a final region. -/
739 def Covered (X : Atl) (x : X.E) (t : X.G.obj x) : Prop :=
740   storedCovered X x t
741
742 theorem covered_region (X : Atl) (k : TerritoryIndex X) (l : territory X k) :
743   Covered X (X.P.cell X.P.folio.lastBase k) l :=
744   <|k, l, rfl|>
745
746 /-- The order and coverage conditions for atlas traversals. -/
747 def IsTraversal : MorphismProperty Atl := fun X Y F =>
748   Function.Injective F.P.obj /\
749   (forall x y, elementLT _ x y -> elementLT _ (F.P.obj x) (F.P.obj y)) /\
750   (forall x (t : X.G.obj x), Covered X x t ->
751     Covered Y (F.P.obj x) (F.A.app x t))
752
753 instance : IsTraversal.IsMultiplicative where
754   id_mem X := by
755     refine <|Function.injective_id, fun _ _ h => h, ?_|>
756     intro x t h
757     simpa [AtlHom.identity] using h
758   comp_mem f g hf hg := by
759     refine <|hg.1.comp hf.1, fun x y h => hg.2.1 _ _ (hf.2.1 _ _ h), ?_|>
760     intro x t h
761     simpa [AtlHom.comp] using hg.2.2 (f.P.obj x) (f.A.app x t) (hf.2.2 x t h)
762
763 abbrev AtlTrav := WideSubcategory IsTraversal
764
765 def AtlTravInc : AtlTrav ==> Atl := wideSubcategoryInclusion IsTraversal
766
767 def AtlTravToAtlTrap : AtlTrav ==> AtlTrap where
768   obj X := WideSubcategory.mk X.obj
769   map f := <|f.1, f.2.1|>
770
771 def IsStableTraversal : MorphismProperty AtlTrav := fun X Y F =>
772   F.1.P.obj X.obj.P.folio.originElement = Y.obj.P.folio.originElement
773
774 instance : IsStableTraversal.IsMultiplicative where
775   id_mem _ := rfl
776   comp_mem f g hf hg := by
777     change g.1.P.obj (f.1.P.obj _) = _
778     rw [hf, hg]
779
780 abbrev StaAtlTrav := WideSubcategory IsStableTraversal
781
782 def StaAtlTravToAtlTrav : StaAtlTrav ==> AtlTrav :=
783   wideSubcategoryInclusion IsStableTraversal
784
785 def StaAtlTravInc : StaAtlTrav ==> Atl := StaAtlTravToAtlTrav then AtlTravInc
786
787 /-! `StableAtlasFamily` implements atlas federations of stable atlases.
788 The empty family is the empty atlas operation, and concatenation retains all
789 components without choosing representatives or identifying their data. Most
790 importantly, every component arrow is an arrow of `StaAtlTrav`, so stability is
791 checked by Lean at the boundary of the horizontal construction. -/
792
793 structure StableAtlasFamily where
794   Index : Type
795   countableIndex : Countable Index
796   component : Index -> StaAtlTrav

```

```

797
798 attribute [instance] StableAtlasFamily.countableIndex
799
800 structure StableAtlasFamilyHom (X Y : StableAtlasFamily) where
801   index : X.Index -> Y.Index
802   component : forall i, X.component i --> Y.component (index i)
803
804 /-- Each component of an atlas-federation morphism is stable by construction. -/
805 theorem StableAtlasFamilyHom.component_stable {X Y : StableAtlasFamily}
806   (f : StableAtlasFamilyHom X Y) (i : X.Index) :
807     IsStableTraversal (f.component i).1 :=
808     (f.component i).2
809
810 @[ext]
811 theorem StableAtlasFamilyHom.ext {X Y : StableAtlasFamily}
812   (f g : StableAtlasFamilyHom X Y) (hi : f.index = g.index)
813   (hc : forall i, HEq (f.component i) (g.component i)) : f = g := by
814   cases f
815   cases g
816   cases hi
817   congr
818   funext i
819   exact eq_of_heq (hc i)
820
821 def StableAtlasFamilyHom.identity (X : StableAtlasFamily) :
822   StableAtlasFamilyHom X X where
823   index := id
824   component := fun _ => Id _
825
826 def StableAtlasFamilyHom.comp {X Y Z : StableAtlasFamily}
827   (f : StableAtlasFamilyHom X Y) (g : StableAtlasFamilyHom Y Z) :
828   StableAtlasFamilyHom X Z where
829   index := g.index o f.index
830   component := fun i => f.component i >> g.component (f.index i)
831
832 instance : Category StableAtlasFamily where
833   Hom := StableAtlasFamilyHom
834   id := StableAtlasFamilyHom.identity
835   comp := StableAtlasFamilyHom.comp
836   id_comp f := by
837     apply StableAtlasFamilyHom.ext
838     . rfl
839     . intro i
840     exact heq_of_eq (Category.id_comp _)
841   comp_id f := by
842     apply StableAtlasFamilyHom.ext
843     . rfl
844     . intro i
845     exact heq_of_eq (Category.comp_id _)
846   assoc f g h := by
847     apply StableAtlasFamilyHom.ext
848     . rfl
849     . intro i
850     exact heq_of_eq (Category.assoc _ _ _)
851
852 @[simp]
853 theorem StableAtlasFamilyHom.component_id (X : StableAtlasFamily)
854   (i : X.Index) :
855   (Id X : X --> X).component i = Id (X.component i) := rfl
856
857 @[simp]
858 theorem StableAtlasFamilyHom.component_comp {X Y Z : StableAtlasFamily}
859   (f : X --> Y) (g : Y --> Z) (i : X.Index) :
860   (f >> g).component i = f.component i >> g.component (f.index i) := rfl
861
862 /-- An atlas as a single-component atlas federation. -/
863 def stableAtlasAtom (X : StaAtlTrav) : StableAtlasFamily where
864   Index := Unit
865   countableIndex := inferInstance
866   component := fun _ => X
867
868 /-- The empty atlas federation has no components. -/
869 def stableAtlasUnit : StableAtlasFamily where

```

```

870   Index := Empty
871   countableIndex := inferInstance
872   component := fun i => nomatch i
873
874   /-- Federation merge is disjoint tagged retention of components,
875   not a categorical coproduct of atlases. -/
876   def stableAtlasMerge (X Y : StableAtlasFamily) : StableAtlasFamily where
877     Index := Sum X.Index Y.Index
878     countableIndex := inferInstance
879     component := Sum.elim X.component Y.component
880
881   def stableAtlasMergeHom {X_1 X_2 Y_1 Y_2 : StableAtlasFamily}
882     (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
883     stableAtlasMerge X_1 X_2 --> stableAtlasMerge Y_1 Y_2 where
884     index := Sum.map f.index g.index
885     component
886     | .inl i => f.component i
887     | .inr j => g.component j
888
889   /-- Horizontal sum on atlas federations. -/
890   def StableAtlHorSum : StableAtlasFamily * StableAtlasFamily ==>
891     StableAtlasFamily where
892     obj X := stableAtlasMerge X.1 X.2
893     map f := stableAtlasMergeHom f.1 f.2
894     map_id X := by
895       apply StableAtlasFamilyHom.ext
896       . funext i
897       cases i <|> rfl
898       . intro i
899       cases i <|> exact HEq.rfl
900     map_comp f g := by
901       apply StableAtlasFamilyHom.ext
902       . funext i
903       cases i <|> rfl
904       . intro i
905       cases i <|> exact HEq.rfl
906
907   def stableAtlasAssociatorHom (X Y Z : StableAtlasFamily) :
908     stableAtlasMerge (stableAtlasMerge X Y) Z -->
909     stableAtlasMerge X (stableAtlasMerge Y Z) where
910     index
911     | .inl (.inl i) => .inl i
912     | .inl (.inr j) => .inr (.inl j)
913     | .inr k => .inr (.inr k)
914     component
915     | .inl (.inl i) => Id (X.component i)
916     | .inl (.inr j) => Id (Y.component j)
917     | .inr k => Id (Z.component k)
918
919   def stableAtlasAssociatorInv (X Y Z : StableAtlasFamily) :
920     stableAtlasMerge X (stableAtlasMerge Y Z) -->
921     stableAtlasMerge (stableAtlasMerge X Y) Z where
922     index
923     | .inl i => .inl (.inl i)
924     | .inr (.inl j) => .inl (.inr j)
925     | .inr (.inr k) => .inr k
926     component
927     | .inl i => Id (X.component i)
928     | .inr (.inl j) => Id (Y.component j)
929     | .inr (.inr k) => Id (Z.component k)
930
931   def stableAtlasAssociator (X Y Z : StableAtlasFamily) :
932     stableAtlasMerge (stableAtlasMerge X Y) Z Iso
933     stableAtlasMerge X (stableAtlasMerge Y Z) where
934     hom := stableAtlasAssociatorHom X Y Z
935     inv := stableAtlasAssociatorInv X Y Z
936     hom_inv_id := by
937       apply StableAtlasFamilyHom.ext
938       . funext i
939       rcases i with (i | j) | k <|> rfl
940       . intro i
941       rcases i with (i | j) | k <|>
942       simp [stableAtlasAssociatorHom, stableAtlasAssociatorInv,

```

```

943     stableAtlasMerge]
944   inv_hom_id := by
945     apply StableAtlasFamilyHom.ext
946     . funext i
947       rcases i with i | (j | k) <;> rfl
948     . intro i
949       rcases i with i | (j | k) <;>
950       simp [stableAtlasAssociatorHom, stableAtlasAssociatorInv,
951         stableAtlasMerge]
952
953   def stableAtlasLeftUnitorHom (X : StableAtlasFamily) :
954     stableAtlasMerge stableAtlasUnit X --> X where
955     index
956       | .inr i => i
957     component
958       | .inr i => Id (X.component i)
959
960   def stableAtlasLeftUnitorInv (X : StableAtlasFamily) :
961     X --> stableAtlasMerge stableAtlasUnit X where
962     index i := .inr i
963     component i := Id (X.component i)
964
965   def stableAtlasLeftUnitor (X : StableAtlasFamily) :
966     stableAtlasMerge stableAtlasUnit X Iso X where
967     hom := stableAtlasLeftUnitorHom X
968     inv := stableAtlasLeftUnitorInv X
969     hom_inv_id := by
970       apply StableAtlasFamilyHom.ext
971       . funext i
972         rcases i with i | i
973         . exact i.elim
974         . rfl
975       . intro i
976         rcases i with i | i
977         . exact i.elim
978         . simp [stableAtlasLeftUnitorHom, stableAtlasLeftUnitorInv,
979           stableAtlasMerge]
980     inv_hom_id := by
981       apply StableAtlasFamilyHom.ext
982       . rfl
983       . intro i
984       simp [stableAtlasLeftUnitorHom, stableAtlasLeftUnitorInv,
985         stableAtlasMerge]
986
987   def stableAtlasRightUnitorHom (X : StableAtlasFamily) :
988     stableAtlasMerge X stableAtlasUnit --> X where
989     index
990       | .inl i => i
991     component
992       | .inl i => Id (X.component i)
993
994   def stableAtlasRightUnitorInv (X : StableAtlasFamily) :
995     X --> stableAtlasMerge X stableAtlasUnit where
996     index i := .inl i
997     component i := Id (X.component i)
998
999   def stableAtlasRightUnitor (X : StableAtlasFamily) :
1000     stableAtlasMerge X stableAtlasUnit Iso X where
1001     hom := stableAtlasRightUnitorHom X
1002     inv := stableAtlasRightUnitorInv X
1003     hom_inv_id := by
1004       apply StableAtlasFamilyHom.ext
1005       . funext i
1006         rcases i with i | i
1007         . rfl
1008         . exact i.elim
1009       . intro i
1010         rcases i with i | i
1011         . simp [stableAtlasRightUnitorHom, stableAtlasRightUnitorInv,
1012           stableAtlasMerge]
1013         . exact i.elim
1014     inv_hom_id := by
1015       apply StableAtlasFamilyHom.ext

```

```

1016   . rfl
1017   . intro i
1018   simp [stableAtlasRightUnitorHom, stableAtlasRightUnitorInv,
1019         stableAtlasMerge]
1020
1021 def stableAtlasBraidingHom (X Y : StableAtlasFamily) :
1022   stableAtlasMerge X Y --> stableAtlasMerge Y X where
1023   index
1024   | .inl i => .inr i
1025   | .inr j => .inl j
1026   component
1027   | .inl i => Id (X.component i)
1028   | .inr j => Id (Y.component j)
1029
1030 def stableAtlasBraiding (X Y : StableAtlasFamily) :
1031   stableAtlasMerge X Y Iso stableAtlasMerge Y X where
1032   hom := stableAtlasBraidingHom X Y
1033   inv := stableAtlasBraidingHom Y X
1034   hom_inv_id := by
1035     apply StableAtlasFamilyHom.ext
1036     . funext i
1037     cases i <;> rfl
1038     . intro i
1039     cases i <;> simp [stableAtlasBraidingHom, stableAtlasMerge]
1040   inv_hom_id := by
1041     apply StableAtlasFamilyHom.ext
1042     . funext i
1043     cases i <;> rfl
1044     . intro i
1045     cases i <;> simp [stableAtlasBraidingHom, stableAtlasMerge]
1046
1047 instance stableAtlasMonoidalStruct : MonoidalCategoryStruct StableAtlasFamily where
1048   tensorObj := stableAtlasMerge
1049   tensorHom := stableAtlasMergeHom
1050   whiskerLeft X _ f := stableAtlasMergeHom (Id X) f
1051   whiskerRight f Y := stableAtlasMergeHom f (Id Y)
1052   tensorUnit := stableAtlasUnit
1053   associator := stableAtlasAssociator
1054   leftUnitor := stableAtlasLeftUnitor
1055   rightUnitor := stableAtlasRightUnitor
1056
1057 instance stableAtlasMonoidal : MonoidalCategory StableAtlasFamily :=
1058   MonoidalCategory.ofTensorHom
1059   (id_tensorHom_id := fun X Y => StableAtHorSum.map_id (X, Y))
1060   (id_tensorHom := fun X { _ } f => rfl)
1061   (tensorHom_id := fun { _ } f Y => rfl)
1062   (tensorHom_comp_tensorHom := fun {X_1 Y_1 Z_1 X_2 Y_2 Z_2} f_1 f_2 g_1 g_2 => by
1063     apply StableAtlasFamilyHom.ext
1064     . funext i
1065     cases i <;> rfl
1066     . intro i
1067     cases i <;> simp [stableAtlasMonoidalStruct, stableAtlasMergeHom])
1068   (associator_naturality := fun { _ _ _ _ } f_1 f_2 f_3 => by
1069     apply StableAtlasFamilyHom.ext
1070     . funext i
1071     rcases i with (i | j) | k <;> rfl
1072     . intro i
1073     rcases i with (i | j) | k <;>
1074       simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1075             stableAtlasAssociator, stableAtlasAssociatorHom, stableAtlasMerge])
1076   (leftUnitor_naturality := fun { _ _ } f => by
1077     apply StableAtlasFamilyHom.ext
1078     . funext i
1079     rcases i with i | i
1080     . exact i.elim
1081     . rfl
1082     . intro i
1083     rcases i with i | i
1084     . exact i.elim
1085     . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1086             stableAtlasLeftUnitor, stableAtlasLeftUnitorHom, stableAtlasMerge])
1087   (rightUnitor_naturality := fun { _ _ } f => by
1088     apply StableAtlasFamilyHom.ext

```

```

1089     . funext i
1090     rcases i with i | i
1091     . rfl
1092     . exact i.elim
1093   . intro i
1094   rcases i with i | i
1095   . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1096     stableAtlasRightUnitor, stableAtlasRightUnitorHom, stableAtlasMerge]
1097   . exact i.elim
1098 (pentagon := fun W X Y Z => by
1099   apply StableAtlasFamilyHom.ext
1100   . funext i
1101   rcases i with ((i | j) | k) | l <;> rfl
1102   . intro i
1103   rcases i with ((i | j) | k) | l <;>
1104     simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1105       stableAtlasAssociator, stableAtlasAssociatorHom, stableAtlasMerge])
1106 (triangle := fun X Y => by
1107   apply StableAtlasFamilyHom.ext
1108   . funext i
1109   rcases i with (i | e) | j
1110   . rfl
1111   . exact e.elim
1112   . rfl
1113   . intro i
1114   rcases i with (i | e) | j
1115   . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1116     stableAtlasAssociator, stableAtlasAssociatorHom,
1117     stableAtlasRightUnitor, stableAtlasRightUnitorHom, stableAtlasMerge]
1118   . exact e.elim
1119   . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1120     stableAtlasAssociator, stableAtlasAssociatorHom,
1121     stableAtlasLeftUnitor, stableAtlasLeftUnitorHom, stableAtlasMerge])
1122
1123 @[simp]
1124 theorem stableAtlas_tensorObj (X Y : StableAtlasFamily) :
1125   MonoidalCategoryStruct.tensorObj X Y = stableAtlasMerge X Y := rfl
1126
1127 @[simp]
1128 theorem stableAtlas_tensorHom {X_1 Y_1 X_2 Y_2 : StableAtlasFamily}
1129   (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
1130   MonoidalCategoryStruct.tensorHom f g = stableAtlasMergeHom f g := rfl
1131
1132 @[simp]
1133 theorem stableAtlas_whiskerLeft (X : StableAtlasFamily)
1134   {Y_1 Y_2 : StableAtlasFamily} (f : Y_1 --> Y_2) :
1135   MonoidalCategoryStruct.whiskerLeft X f =
1136     stableAtlasMergeHom (Id X) f := rfl
1137
1138 @[simp]
1139 theorem stableAtlas_whiskerRight {X_1 X_2 : StableAtlasFamily}
1140   (f : X_1 --> X_2) (Y : StableAtlasFamily) :
1141   MonoidalCategoryStruct.whiskerRight f Y =
1142     stableAtlasMergeHom f (Id Y) := rfl
1143
1144 @[simp]
1145 theorem stableAtlas_associator (X Y Z : StableAtlasFamily) :
1146   MonoidalCategoryStruct.associator X Y Z =
1147     stableAtlasAssociator X Y Z := rfl
1148
1149 instance stableAtlasBraided : BraidedCategory StableAtlasFamily where
1150   braiding := stableAtlasBraiding
1151   braiding_naturality_right := fun X { _ _ } f => by
1152     apply StableAtlasFamilyHom.ext
1153     . funext i
1154     cases i <;> rfl
1155     . intro i
1156     cases i <;>
1157       simp [stableAtlasMergeHom, stableAtlasBraiding,
1158         stableAtlasBraidingHom, stableAtlasMerge]
1159   braiding_naturality_left := fun { _ _ } f Z => by
1160     apply StableAtlasFamilyHom.ext
1161     . funext i

```

```

1162     cases i <;> rfl
1163   . intro i
1164     cases i <;>
1165       simp [stableAtlasMergeHom, stableAtlasBraiding,
1166             stableAtlasBraidingHom, stableAtlasMerge]
1167   hexagon_forward := fun X Y Z => by
1168     apply StableAtlasFamilyHom.ext
1169   . funext i
1170     rcases i with (i | j) | k <;> rfl
1171   . intro i
1172     rcases i with (i | j) | k <;>
1173       simp [stableAtlasMergeHom, stableAtlasAssociator,
1174             stableAtlasAssociatorHom, stableAtlasBraiding,
1175             stableAtlasBraidingHom, stableAtlasMerge]
1176   hexagon_reverse := fun X Y Z => by
1177     apply StableAtlasFamilyHom.ext
1178   . funext i
1179     rcases i with i | (j | k) <;> rfl
1180   . intro i
1181     rcases i with i | (j | k) <;>
1182       simp [stableAtlasMergeHom, stableAtlasAssociator,
1183             stableAtlasAssociatorInv, stableAtlasBraiding,
1184             stableAtlasBraidingHom, stableAtlasMerge]
1185
1186 instance : SymmetricCategory StableAtlasFamily where
1187   symmetry X Y := by
1188     apply StableAtlasFamilyHom.ext
1189   . funext i
1190     cases i <;> rfl
1191   . intro i
1192     cases i <;>
1193       simp [stableAtlasBraided, stableAtlasBraiding,
1194             stableAtlasBraidingHom, stableAtlasMerge]
1195
1196 def IsAtlasMap : ObjectProperty Atl := fun A =>
1197   forall v : extent A, Covered A A.P.folio.originElement v
1198
1199 abbrev AtlMap := IsAtlasMap.FullSubcategory
1200
1201 def AtlMapInc : AtlMap ==> Atl := ObjectProperty.iota IsAtlasMap
1202
1203 def IsAtlasMapTrav : ObjectProperty AtlTrav := fun A => IsAtlasMap A.obj
1204 abbrev AtlTravMap := IsAtlasMapTrav.FullSubcategory
1205
1206 def AtlTravMapInc : AtlTravMap ==> AtlTrav := ObjectProperty.iota IsAtlasMapTrav
1207
1208 /-- The dominion of covered data in a cell. -/
1209 def coveredDom (X : Atl) (x : X.E) : DomIns where
1210   toDom :=
1211     { Carrier := {t : X.G.obj x // Covered X x t}
1212     rank :=
1213       ({ toFun := Subtype.val
1214         inj' := Subtype.val_injective } :
1215         Function.Embedding {t : X.G.obj x // Covered X x t} (X.G.obj x)).trans
1216         (X.G.obj x).toDom.rank }
1217
1218 def coveredMap {X : Atl} {x y : X.E} (f : x --> y) :
1219   coveredDom X x --> coveredDom X y where
1220   toFun t := <|X.G.map f t.1, by
1221     rcases t.2 with <|k, l, h|>
1222     refine <|k, l, ?_|>
1223     have e : f >> X.P.folio.toOrigin y = X.P.folio.toOrigin x :=
1224       CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
1225     change (X.G.map f >> X.G.map (X.P.folio.toOrigin y)) t.1 = _
1226     rw [<- X.G.map_comp, e]
1227     exact h|>
1228   inj' := fun a b h => Subtype.ext <| (X.G.map f).injective <| congrArg Subtype.val h
1229
1230 def coveredFunctor (X : Atl) : X.E ==> DomIns where
1231   obj := coveredDom X
1232   map := coveredMap
1233   map_id x := by
1234     apply DomIns.hom_ext

```

```

1235     intro t
1236     apply Subtype.ext
1237     change (X.G.map (Id x)) t.1 = t.1
1238     rw [X.G.map_id]
1239     change Function.Embedding.refl _ t.1 = t.1
1240     rfl
1241 map_comp f g := by
1242   apply DomIns.hom_ext
1243   intro t
1244   apply Subtype.ext
1245   change (X.G.map (f >> g)) t.1 =
1246     (X.G.map f >> X.G.map g) t.1
1247   rw [X.G.map_comp]
1248
1249 def chartAtlas (X : Atl) : Atl where
1250   P := X.P
1251   G := coveredFunctor X
1252   disjoint := by
1253     intro m i j hij x y h
1254     apply X.disjoint m i j hij x.1 y.1
1255     exact congrArg Subtype.val h
1256
1257 theorem chart_all_covered (X : Atl) (x : (chartAtlas X).E)
1258   (t : (chartAtlas X).G.obj x) : Covered (chartAtlas X) x t := by
1259   rcases t.2 with <|k, l, h|>
1260   let l' : territory (chartAtlas X) k := <|l, covered_region X k l|>
1261   exact <|k, l', Subtype.ext h|>
1262
1263 theorem chart_isAtlasMap (X : Atl) : IsAtlasMap (chartAtlas X) :=
1264   fun v => chart_all_covered X _ v
1265
1266 def chartCounit (X : Atl) : chartAtlas X --> X where
1267   P := Unit X.E
1268   A :=
1269     { app := fun _ =>
1270       { toFun := Subtype.val
1271         inj' := Subtype.val_injective }
1272       naturality := by intros; ext; rfl }
1273
1274 def chartMap {X Y : AtlTrav} (f : X --> Y) :
1275   chartAtlas X.obj --> chartAtlas Y.obj where
1276   P := f.1.P
1277   A :=
1278     { app := fun x =>
1279       { toFun := fun t => <|f.1.A.app x t.1, f.2.2.2 x t.1 t.2|>
1280         inj' := fun a b h => Subtype.ext <| (f.1.A.app x).injective <|
1281           congrArg Subtype.val h }
1282       naturality := by
1283         intro x y g
1284         apply DomIns.hom_ext
1285         intro t
1286         apply Subtype.ext
1287         exact congrFun (congrArg Function.Embedding.toFun (f.1.A.naturality g)) t.1 }
1288
1289 theorem chartMap_isTraversal {X Y : AtlTrav} (f : X --> Y) :
1290   IsTraversal (chartMap f) := by
1291   refine <|f.2.1, f.2.2.1, ?_|>
1292   intro x t _
1293   exact chart_all_covered Y.obj _ _
1294
1295 def Chr : AtlTrav ==> AtlTravMap where
1296   obj X := <|WideSubcategory.mk (chartAtlas X.obj), chart_isAtlasMap X.obj|>
1297   map f := <|chartMap f, chartMap_isTraversal f|>
1298   map_id _ := by
1299     apply Subtype.ext
1300     change chartMap (Id _) = AtlHom.identity _
1301     apply AtlHom.ext
1302     . rfl
1303     . apply heq_of_eq
1304     ext x t
1305     rfl
1306   map_comp f g := by
1307     apply Subtype.ext

```



```

1308   change chartMap (_ >> _) = AtlHom.comp (chartMap _) (chartMap _)
1309   apply AtlHom.ext
1310   . rfl
1311   . apply heq_of_eq
1312     ext x t
1313     rfl
1314
1315   /-- The formal construction realizes the proof sketch by retaining in each
1316   cell exactly the data covered by final regions; its counit is the canonical
1317   inclusion into the original atlas. -/
1318
1319   theorem originImage_origin (X : Atl) (v : extent X) :
1320     originImage X X.P.folio.originElement v = v := by
1321     have e : X.P.folio.toOrigin X.P.folio.originElement =
1322       Id X.P.folio.originElement :=
1323       CategoryOfElements.ext _ _ (Subsingleton.elim _ _)
1324     rw [originImage, storedOriginImage, e, X.G.map_id]
1325     change Function.Embedding.refl _ v = v
1326     rfl
1327
1328   theorem atlasMap_all_covered {X : Atl} (hX : IsAtlasMap X)
1329     (x : X.E) (t : X.G.obj x) : Covered X x t := by
1330     rcases hX (originImage X x t) with <|k, l, h|>
1331     exact <|k, l, by
1332       change originImage X X.P.folio.originElement (originImage X x t) =
1333         originImage X (X.P.cell X.P.folio.lastBase k) l at h
1334       simpa only [originImage_origin] using h|>
1335
1336   theorem chartCounit_isTraversal (X : Atl) : IsTraversal (chartCounit X) := by
1337     refine <|Function.injective_id, fun _ _ h => h, ?_|>
1338     intro x t _
1339     simpa [chartCounit] using t.2
1340
1341   def chartCounitTrav (X : AtlTrav) :
1342     (WideSubcategory.mk (chartAtlas X.obj) : AtlTrav) --> X :=
1343     <|chartCounit X.obj, chartCounit_isTraversal X.obj|>
1344
1345   def chartLift {A : AtlTravMap} {X : AtlTrav}
1346     (f : AtlTravMapInc.obj A --> X) :
1347     A --> Chr.obj X := by
1348     refine <|?, ?_|>
1349     . exact
1350       { P := f.1.P
1351       A :=
1352         { app := fun x =>
1353           { toFun := fun t => <|f.1.A.app x t,
1354             f.2.2.2 x t (atlasMap_all_covered A.property x t)|>
1355           inj' := by
1356             intro a b h
1357             apply (f.1.A.app x).injective
1358             exact congrArg (fun z => z.1) h }
1359         naturality := by
1360           intro x y g
1361           apply DomIns.hom_ext
1362           intro t
1363           apply Subtype.ext
1364           exact congrFun (congrArg Function.Embedding.toFun (f.1.A.naturality g)) t } }
1365     . refine <|f.2.1, f.2.2.1, ?_|>
1366     intro x t _
1367     exact chart_all_covered X.obj _ _
1368
1369   def chartHomEquiv (A : AtlTravMap) (X : AtlTrav) :
1370     (AtlTravMapInc.obj A --> X) Equiv (A --> Chr.obj X) where
1371     toFun := chartLift
1372     invFun g := g >> chartCounitTrav X
1373     left_inv f := by
1374       apply Subtype.ext
1375       apply AtlHom.ext
1376       . rfl
1377       . apply heq_of_eq
1378         ext x t
1379         rfl
1380     right_inv g := by

```

```

1381   apply Subtype.ext
1382   apply AtlHom.ext
1383   . rfl
1384   . apply heq_of_eq
1385     ext x t
1386     apply Subtype.ext
1387     rfl
1388
1389   theorem chartHomEquiv_naturality_left {A A' : AtlTravMap} (f : A' --> A)
1390     {X : AtlTrav} (g : AtlTravMapInc.obj A --> X) :
1391     chartHomEquiv A' X (AtlTravMapInc.map f >> g) =
1392     f >> chartHomEquiv A X g := by
1393   apply Subtype.ext
1394   apply AtlHom.ext
1395   . rfl
1396   . apply heq_of_eq
1397     ext x t
1398     apply Subtype.ext
1399     rfl
1400
1401   theorem chartHomEquiv_naturality_right {A : AtlTravMap} {X X' : AtlTrav}
1402     (f : AtlTravMapInc.obj A --> X) (g : X --> X') :
1403     chartHomEquiv A X' (f >> g) = chartHomEquiv A X f >> Chr.map g := by
1404   apply Subtype.ext
1405   apply AtlHom.ext
1406   . rfl
1407   . apply heq_of_eq
1408     ext x t
1409     apply Subtype.ext
1410     rfl
1411
1412   def cartographyAdjunction : AtlTravMapInc -| Chr :=
1413     Adjunction.mkOfHomEquiv
1414     { homEquiv := chartHomEquiv
1415       homEquiv_naturality_left_symm := by
1416         intro X' X Y f g
1417         apply (chartHomEquiv X' Y).injective
1418         simpa using (chartHomEquiv_naturality_left f ((chartHomEquiv X Y).symm g)).symm
1419       homEquiv_naturality_right := by
1420         intro X Y Y' f g
1421         exact chartHomEquiv_naturality_right f g }
1422
1423   theorem cartographyLemma : Nonempty (AtlTravMapInc -| Chr) :=
1424     <|cartographyAdjunction|>
1425
1426   def extentMap {X Y : Atl} (f : X --> Y) : extent X --> extent Y :=
1427     f.A.app X.P.folio.originElement >>
1428     Y.G.map (Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement))
1429
1430   theorem extentMap_originImage {X Y : Atl} (f : X --> Y) (x : X.E)
1431     (a : X.G.obj x) :
1432     extentMap f (originImage X x a) =
1433     originImage Y (f.P.obj x) (f.A.app x a) := by
1434   simp only [extentMap, originImage, storedOriginImage]
1435   have hn := f.A.naturality (X.P.folio.toOrigin x)
1436   have hc : f.P.map (X.P.folio.toOrigin x) >>
1437     Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement) =
1438     Y.P.folio.toOrigin (f.P.obj x) := by
1439     apply CategoryOfElements.ext
1440     apply Subsingleton.elim
1441   rw [← hc, Y.G.map_comp]
1442   exact congrFun (congrArg Function.Embedding.toFun
1443     (congrArg (fun k => k >>
1444       Y.G.map (Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement)))) hn) a
1445
1446   theorem extentMap_id (X : Atl) : extentMap (Id X) = Id (extent X) := by
1447   apply DomIns.hom_ext
1448   intro t
1449   change (X.G.map (X.P.folio.toOrigin X.P.folio.originElement)) t = t
1450   simpa [originImage] using originImage_origin X t
1451
1452   theorem extentMap_comp {X Y Z : Atl} (f : X --> Y) (g : Y --> Z) :
1453     extentMap (f >> g) = extentMap f >> extentMap g := by

```

```

1454 apply DomIns.hom_ext
1455 intro t
1456 let oX := X.P.folio.originElement
1457 let y := f.P.obj oX
1458 let oY := Y.P.folio.originElement
1459 let z := g.P.obj y
1460 let z_0 := g.P.obj oY
1461 have hn := g.A.naturality (Y.P.folio.toOrigin y)
1462 have hz : g.P.map (Y.P.folio.toOrigin y) >> Z.P.folio.toOrigin z_0 =
1463   Z.P.folio.toOrigin z :=
1464     CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
1465 have hn' : Y.G.map (Y.P.folio.toOrigin y) >> g.A.app oY =
1466   g.A.app y >> Z.G.map (g.P.map (Y.P.folio.toOrigin y)) := hn
1467 have hm : g.A.app y >> Z.G.map (Z.P.folio.toOrigin z) =
1468   Y.G.map (Y.P.folio.toOrigin y) >> g.A.app oY >>
1469     Z.G.map (Z.P.folio.toOrigin z_0) := by
1470   rw [← Category.assoc, hn', Category.assoc, ← Z.G.map_comp, hz]
1471 exact congrFun (congrArg Function.Embedding.toFun
1472   (by simp only [Category.assoc] using congrArg (fun q => f.A.app oX >> q) hm)) t
1473
1474 def Ex : Atl ==> DomIns where
1475   obj := extent
1476   map := extentMap
1477   map_id := extentMap_id
1478   map_comp := extentMap_comp
1479
1480 def stableCoalitionMap {X Y : StaAtlTrav} (f : X --> Y) :
1481   coveredDom X.obj.obj X.obj.obj.P.folio.originElement -->
1482   coveredDom Y.obj.obj Y.obj.obj.P.folio.originElement where
1483   toFun t := by
1484     have hc := f.1.2.2.2 X.obj.obj.P.folio.originElement t.1 t.2
1485     exact coveredMap (X := Y.obj.obj) (eqToHom f.2)
1486       <|f.1.1.A.app X.obj.obj.P.folio.originElement t.1, hc|>
1487   inj' := fun a b h => by
1488     apply Subtype.ext
1489     apply (f.1.1.A.app X.obj.obj.P.folio.originElement).injective
1490     apply (Y.obj.obj.G.map (eqToHom f.2)).injective
1491     exact congrArg Subtype.val h
1492
1493 theorem stableCoalitionMap_val {X Y : StaAtlTrav} (f : X --> Y)
1494   (t : coveredDom X.obj.obj X.obj.obj.P.folio.originElement) :
1495   (stableCoalitionMap f t).1 = extentMap f.1.1 t.1 := by
1496   change Y.obj.obj.G.map (eqToHom f.2)
1497     (f.1.1.A.app X.obj.obj.P.folio.originElement t.1) =
1498     Y.obj.obj.G.map (Y.obj.obj.P.folio.toOrigin
1499       (f.1.1.P.obj X.obj.obj.P.folio.originElement))
1500     (f.1.1.A.app X.obj.obj.P.folio.originElement t.1)
1501   have hmor : eqToHom f.2 = Y.obj.obj.P.folio.toOrigin
1502     (f.1.1.P.obj X.obj.obj.P.folio.originElement) := by
1503     apply CategoryOfElements.ext
1504     apply Subsingleton.elim
1505   rw [hmo]
1506
1507 /-- `Coa` is written directly on the covered origins. This is definitionally
1508 the object part of `Ex o Chr`; stability makes its action on arrows reduce to
1509 the component at the origin. -/
1510 def Coa : StaAtlTrav ==> DomIns where
1511   obj X := coveredDom X.obj.obj X.obj.obj.P.folio.originElement
1512   map := stableCoalitionMap
1513   map_id X := by
1514     apply DomIns.hom_ext
1515     intro t
1516     apply Subtype.ext
1517     rw [stableCoalitionMap_val]
1518     exact congrFun (congrArg Function.Embedding.toFun (extentMap_id X.obj.obj)) t.1
1519   map_comp f g := by
1520     apply DomIns.hom_ext
1521     intro t
1522     apply Subtype.ext
1523     rw [stableCoalitionMap_val]
1524     change extentMap (f.1.1 >> g.1.1) t.1 =
1525       (stableCoalitionMap g (stableCoalitionMap f t)).1
1526     rw [stableCoalitionMap_val, stableCoalitionMap_val]

```

```

1527     exact congrFun (congrArg Function.Embedding.toFun
1528       (extentMap_comp f.1.1 g.1.1)) t.1
1529
1530 def coalition (X : Atl) : DomIns := (Coa.obj
1531   (WideSubcategory.mk (WideSubcategory.mk X) : StaAtlTrav))
1532
1533 /-- The initial dominion. -/
1534 def emptyDominion : DomIns where
1535   toDom :=
1536     { Carrier := Empty
1537       rank :=
1538         { toFun := fun x => nomatch x
1539           inj' := fun x => nomatch x } }
1540
1541 def singletonChain : Chain where
1542   Obj := Unit
1543   linearOrder := inferInstance
1544   wellFoundedLT := inferInstance
1545   orderType_lt := by
1546     have hpow : Ordinal.omega0 ^ (1 : Ordinal) <
1547       Ordinal.omega0 ^ Ordinal.omega0 :=
1548       (Ordinal.opow_lt_opow_iff_right Ordinal.one_lt_omega0).2
1549     Ordinal.one_lt_omega0
1550   have homega : Ordinal.omega0 < Ordinal.omega0 ^ Ordinal.omega0 := by
1551     simp only [Ordinal.opow_one] using hpow
1552     simp using lt_trans Ordinal.one_lt_omega0 homega
1553
1554 def singletonObjEquiv : singletonChain.Obj Equiv Unit := Equiv.refl _
1555
1556 def onePageFolio : Folio where
1557   length := 1
1558   positive := by omega
1559   core := (Functor.const (Fin 1)).obj (op singletonChain)
1560   originEquiv := singletonObjEquiv
1561
1562 def onePagePag : Pag := <|onePageFolio|>
1563
1564 theorem onePage_cell_unique (x : onePagePag.E) :
1565   x = onePageFolio.originElement := by
1566   let hbase : x.1 = onePageFolio.originBase := by
1567     apply unop_injective
1568     apply Fin.eq_zero
1569   apply Functor.Elements.ext x onePageFolio.originElement hbase
1570   exact onePageFolio.origin_unique _
1571
1572 def dominionAtlas (X : DomIns) : Atl where
1573   P := onePagePag
1574   G := (Functor.const onePagePag.E).obj X
1575   disjoint := by
1576     intro m i j hij
1577     exfalso
1578     apply hij
1579     apply onePageFolio.originEquiv.injective
1580     exact Subsingleton.elim _ _
1581
1582 theorem dominionAtlas_isMap (X : DomIns) : IsAtlasMap (dominionAtlas X) := by
1583   intro v
1584   let k : TerritoryIndex (dominionAtlas X) :=
1585     onePageFolio.originValue
1586   refine <|k, v, ?_|>
1587   change originImage (dominionAtlas X)
1588   (dominionAtlas X).P.folio.originElement v =
1589     originImage (dominionAtlas X)
1590     ((dominionAtlas X).P.cell (dominionAtlas X).P.folio.lastBase k) v
1591   rw [originImage_origin]
1592   have hc : (dominionAtlas X).P.cell (dominionAtlas X).P.folio.lastBase k =
1593     (dominionAtlas X).P.folio.originElement := onePage_cell_unique _
1594   rw [hc, originImage_origin]
1595
1596 def dominionMap {X Y : DomIns} (f : X --> Y) :
1597   dominionAtlas X --> dominionAtlas Y where
1598   P := Unit onePagePag.E
1599   A :=

```

```

1600   { app := fun _ => f
1601     naturality := by intros; apply DomIns.hom_ext; intro; rfl }
1602
1603 theorem dominationMap_isTraversal {X Y : DomIns} (f : X --> Y) :
1604   IsTraversal (dominionMap f) := by
1605     refine <|Function.injective_id, fun _ _ h => h, ?_|>
1606     intro x t _
1607     exact atlasMap_all_covered (dominionAtlas_isMap Y) _ _
1608
1609 theorem dominationMap_isStable {X Y : DomIns} (f : X --> Y) :
1610   IsStableTraversal
1611     (X := WideSubcategory.mk (dominionAtlas X))
1612     (Y := WideSubcategory.mk (dominionAtlas Y))
1613     <|dominionMap f, dominationMap_isTraversal f|> := rfl
1614
1615 def DomInc : DomIns ==> StaAtlTrav where
1616   obj X := WideSubcategory.mk (WideSubcategory.mk (dominionAtlas X))
1617   map f := <|<|dominionMap f, dominationMap_isTraversal f|>,
1618     dominationMap_isStable f|>
1619   map_id _ := by
1620     apply Subtype.ext
1621     apply Subtype.ext
1622     apply AtlHom.ext
1623     . rfl
1624     . apply heq_of_eq
1625       ext
1626       rfl
1627   map_comp f g := by
1628     apply Subtype.ext
1629     apply Subtype.ext
1630     apply AtlHom.ext
1631     . rfl
1632     . apply heq_of_eq
1633       ext x t
1634       change (f >> g) t = (f >> g) t
1635     rfl
1636
1637 def onePageToAtlasP (Y : Atl) : onePagePag.E ==> Y.E :=
1638   (Functor.const onePagePag.E).obj Y.P.folio.originElement
1639
1640 theorem onePageToAtlasP_injective (Y : Atl) :
1641   Function.Injective (onePageToAtlasP Y).obj := by
1642     intro x y _
1643     exact onePage_cell_unique x |>.trans (onePage_cell_unique y).symm
1644
1645 theorem onePage_elementLT_false (X : DomIns) (x y : (dominionAtlas X).E) :
1646   not elementLT (dominionAtlas X) x y := by
1647     intro h
1648     have hxy : x = y := onePage_cell_unique x |>.trans (onePage_cell_unique y).symm
1649     subst y
1650     let m := (dominionAtlas X).P.folio.commonBase x.1 x.1
1651     let l : LinearOrder ((dominionAtlas X).H.obj m) :=
1652       ((dominionAtlas X).P.folio.core.obj m.unop).linearOrder
1653     exact lt_irrefl _ h
1654
1655 /-- A stable traversal from a one-page atlas determines an embedding into
1656 the covered part of the target extent. -/
1657 def domIncToCoa {X : DomIns} {Y : StaAtlTrav} (f : DomInc.obj X --> Y) :
1658   X --> Coa.obj Y where
1659   toFun x := by
1660     have hc := f.1.2.2.2 onePageFolio.originElement x
1661     (atlasMap_all_covered (dominionAtlas_isMap X) _ x)
1662     exact coveredMap (X := Y.obj.obj) (eqToHom f.2)
1663     <|f.1.1.A.app onePageFolio.originElement x, hc|>
1664   inj' := fun a b h => by
1665     apply (f.1.1.A.app onePageFolio.originElement).injective
1666     apply (Y.obj.obj.G.map (eqToHom f.2)).injective
1667     exact congrArg Subtype.val h
1668
1669 /-- Conversely, an embedding into the coalition supplies the unique stable
1670 traversal from the corresponding one-page atlas. -/
1671 def coaToDomInc {X : DomIns} {Y : StaAtlTrav} (f : X --> Coa.obj Y) :
1672   DomInc.obj X --> Y := by

```

```

1673 let p : (dominionAtlas X).E ==> Y.obj.obj.P.E := onePageToAtlasP Y.obj.obj
1674 let a : (dominionAtlas X).G --> p then Y.obj.obj.G :=
1675   { app := fun _ =>
1676     { toFun := fun x => (f x).1
1677       inj' := fun x y h => f.injective (Subtype.ext h) }
1678     naturality := by
1679       intro x y g
1680       apply DomIns.hom_ext
1681       intro t
1682       have hxy : x = y := onePage_cell_unique x |>.trans
1683         (onePage_cell_unique y).symm
1684       subst y
1685       have hg : g = Id x := by
1686         apply CategoryOfElements.ext
1687         apply Subsingleton.elim
1688       subst g
1689       simp [p] }
1690 let h : dominionAtlas X --> Y.obj.obj := </p, a/>
1691 have htrav : IsTraversable h := by
1692   refine <|onePageToAtlasP_injective Y.obj.obj, ?_, ?_||>
1693   . intro x y hxy
1694   exact (onePage_elementLT_false X x y hxy).elim
1695   . intro x t _
1696   change Covered Y.obj.obj Y.obj.obj.P.folio.originElement (f t).1
1697   exact (f t).2
1698   exact <|<|h, htrav|>, rfl|>
1699
1700 def coaDomIncIso (X : DomIns) : Coa.obj (DomInc.obj X) Iso X where
1701   hom :=
1702     { toFun := fun x => x.1
1703       inj' := fun x y h => Subtype.ext h }
1704   inv :=
1705     { toFun := fun x => <|x, atlasMap_all_covered
1706       (dominionAtlas_isMap X) onePageFolio.originElement x|>
1707       inj' := fun x y h => congrArg Subtype.val h }
1708   hom_inv_id := by
1709     apply DomIns.hom_ext
1710     intro x
1711     apply Subtype.ext
1712     rfl
1713   inv_hom_id := by
1714     apply DomIns.hom_ext
1715     intro x
1716     rfl
1717
1718 /-- The hom-set equivalence expressing that one-page insertion is left
1719 adjoint to coalization. -/
1720 def domIncCoaHomEquiv (X : DomIns) (Y : StaAtlTrav) :
1721   (DomInc.obj X --> Y) Equiv (X --> Coa.obj Y) where
1722   toFun := domIncToCoa
1723   invFun := coaToDomInc
1724   left_inv := by
1725     intro f
1726     apply Subtype.ext
1727     apply Subtype.ext
1728     have hP : (coaToDomInc (domIncToCoa f)).1.1.P = f.1.1.P := by
1729       exact CategoryTheory.Functor.ext
1730         (fun x => f.2.symm.trans
1731           (congrArg f.1.1.P.obj (onePage_cell_unique x).symm))
1732         (fun _ _ => by
1733           apply CategoryOfElements.ext
1734           apply Subsingleton.elim)
1735     apply AtlHom.ext _ _ hP
1736     apply NatTrans.hext_right _ _
1737     (congrArg (fun Q => Q then Y.obj.obj.G) hP)
1738     intro x
1739     dsimp [coaToDomInc, domIncToCoa, onePageToAtlasP]
1740     have hx : x = onePageFolio.originElement := onePage_cell_unique x
1741     subst x
1742     let e := f.1.1.A.app onePageFolio.originElement
1743     have hrec :
1744       { toFun := fun x =>
1745         (coveredMap (X := Y.obj.obj) (eqToHom f.2)

```

```

1746         <|e x, by
1747             exact f.1.2.2.2 onePageFolio.originElement x
1748             (atlasMap_all_covered (dominionAtlas_isMap X) _ x)|>).1
1749     inj' := by
1750         intro a b hab
1751         apply e.injective
1752         apply (Y.obj.obj.G.map (eqToHom f.2)).injective
1753         exact hab } = e >> Y.obj.obj.G.map (eqToHom f.2) := by
1754     apply DomIns.hom_ext
1755     intro t
1756     rfl
1757     exact (heq_of_eq hrec).trans
1758         (embedding_comp_map_eqToHom_heq Y.obj.obj.G f.2 e)
1759 right_inv := by
1760     intro f
1761     apply DomIns.hom_ext
1762     intro x
1763     apply Subtype.ext
1764     change Y.obj.obj.G.map (Id Y.obj.obj.P.folio.originElement) (f x).1 = (f x).1
1765     exact congrFun (congrArg Function.Embedding.toFun
1766         (Y.obj.obj.G.map_id Y.obj.obj.P.folio.originElement)) (f x).1
1767
1768 theorem domIncCoaHomEquiv_naturality_left {X X' : DomIns} (f : X' --> X)
1769 {Y : StaAtlTrav} (g : DomInc.obj X --> Y) :
1770     domIncCoaHomEquiv X' Y (DomInc.map f >> g) =
1771     f >> domIncCoaHomEquiv X Y g := by
1772     apply DomIns.hom_ext
1773     intro x
1774     apply Subtype.ext
1775     rfl
1776
1777 theorem domIncCoaHomEquiv_naturality_right {X : DomIns} {Y Y' : StaAtlTrav}
1778 (f : DomInc.obj X --> Y) (g : Y --> Y') :
1779     domIncCoaHomEquiv X Y' (f >> g) =
1780     domIncCoaHomEquiv X Y f >> Coa.map g := by
1781     apply DomIns.hom_ext
1782     intro x
1783     apply Subtype.ext
1784     let t : Coa.obj (DomInc.obj X) :=
1785         <|x, atlasMap_all_covered
1786             (dominionAtlas_isMap X) onePageFolio.originElement x|>
1787     change (Coa.map (f >> g) t).1 = (Coa.map g (Coa.map f t)).1
1788     exact congrArg Subtype.val <| congrFun
1789         (congrArg Function.Embedding.toFun (Coa.map_comp f g)) t
1790
1791 /-- The unconditional adjunction promised by the Domanial Inclusion
1792 construction. -/
1793 def dominionAdjunction : DomInc -| Coa :=
1794     Adjunction.mkOfHomEquiv
1795     { homEquiv := domIncCoaHomEquiv
1796       homEquiv_naturality_left_symm := by
1797         intro X' X Y f g
1798         apply (domIncCoaHomEquiv X' Y).injective
1799         simpa using
1800             (domIncCoaHomEquiv_naturality_left f
1801             ((domIncCoaHomEquiv X Y).symm g)).symm
1802       homEquiv_naturality_right := by
1803         intro X Y Y' f g
1804         exact domIncCoaHomEquiv_naturality_right f g }
1805
1806 theorem dominionLemma : Nonempty (DomInc -| Coa) :=
1807     <|dominionAdjunction|>
1808
1809 abbrev DaTra := Atlop ==> Type
1810
1811 def Yo : Atl ==> DaTra := yoneda
1812
1813 /-- A concrete certificate of the statement that `DaTra` is the displayed
1814 presheaf category. -/
1815 def datraToposPresentation : DaTra EquivCat (Atlop ==> Type) :=
1816     CategoryTheory.Equivalence.refl
1817
1818 structure Navigation (D : DaTra) where

```

```

1819 A : Atl
1820 hom : Yo.obj A --> D
1821 mono : Mono hom
1822
1823 attribute [instance] Navigation.mono
1824
1825 structure Expedition (D : DaTra) extends Navigation D where
1826   atlasMap : IsAtlasMap A
1827
1828 abbrev DaTrap := AtlTrapop ==> Type
1829 abbrev DaTrav := AtlTravop ==> Type
1830
1831 /-- Presheaves on the wide category of atlas traversals. -/
1832 abbrev AtlTravPSH := AtlTravop ==> Type
1833
1834 /-- Forget the action of an atlas presheaf on non-traversal arrows. -/
1835 def DaTra.restrictToAtlTrav : DaTra ==> AtlTravPSH :=
1836   (Functor.whiskeringLeft AtlTravop Atlop Type).obj AtlTravInc.op
1837
1838 /-- The promised identification is definitional after restricting the
1839 indexing category. -/
1840 def DaTrav.atlTravPSHEquiv : DaTrav EquivCat AtlTravPSH :=
1841   CategoryTheory.Equivalence.refl
1842
1843 /-- Presheaves on atlas federations of stable atlas traversals. -/
1844 abbrev StaDaTravPresheaf := StableAtlasFamilyop ==> Type 3
1845
1846 /-- The all-objects wrapper gives Day convolution its own monoidal structure,
1847 separate from the pointwise cartesian structure on a raw functor category. -/
1848 def IsStableDataTransversal : ObjectProperty StaDaTravPresheaf := fun _ => True
1849
1850 /-- Stable data transversals: presheaves whose indexing arrows are stable atlas
1851 transversals. Stability is therefore enforced by the source category. -/
1852 abbrev StaDaTrav := IsStableDataTransversal.FullSubcategory
1853
1854 abbrev StaDaTravInc : StaDaTrav ==> StaDaTravPresheaf :=
1855   IsStableDataTransversal.iota
1856
1857 def IsDaTraMap : ObjectProperty DaTra := fun D =>
1858   forall nav : Navigation D, IsAtlasMap nav.A
1859
1860 abbrev DaTraMap := IsDaTraMap.FullSubcategory
1861
1862 def AtlI : Atl := dominionAtlas emptyDominion
1863
1864 theorem AtlI_eq_domInc_zero :
1865   AtlI = (DomInc.obj emptyDominion).obj.obj := rfl
1866
1867 def Folio.pageValue (W : Folio) (m : Fin W.length) :
1868   (W.core.obj m).unop.Obj := by
1869   letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
1870   let q : (W.core.obj m).unop --> (W.core.obj W.originIndex).unop :=
1871     (W.core.map (homOfLE (Fin.zero_le m))).unop
1872   exact Classical.choose (q.point_surjective W.originValue)
1873
1874 theorem Folio.map_unop_comp_apply (W : Folio) {i j k : Fin W.length}
1875   (hik : i --> k) (hij : i --> j) (hjk : j --> k)
1876   (z : (W.core.obj k).unop.Obj) :
1877   (W.core.map hik).unop z =
1878     (W.core.map hij).unop ((W.core.map hjk).unop z) := by
1879   have e : hik = hij >> hjk := Subsingleton.elim _ _
1880   subst hik
1881   have h := congrArg Quiver.Hom.unop (W.core.map_comp hij hjk)
1882   exact congrFun (congrArg ConHom.toFun h) z
1883
1884 def bouquetLength (X Y : Folio) : Nat := max X.length Y.length + 1
1885
1886 def bouquetDepth (X Y : Folio) : Nat := max X.length Y.length
1887
1888 theorem bouquetLength_pos (X Y : Folio) : 0 < bouquetLength X Y := by
1889   simp [bouquetLength]
1890
1891 instance (X Y : Folio) : NeZero (bouquetLength X Y) :=

```



```

1892 <|Nat.ne_of_gt (bouquetLength_pos X Y)|>
1893
1894 def bouquetChain (X Y : Folio) : Fin (bouquetLength X Y) -> Chain :=
1895   Fin.cases singletonChain (fun m : Fin (bouquetDepth X Y) =>
1896     Chain.sum (X.core.obj (X.paddedIndex m.1)).unop
1897       (Y.core.obj (Y.paddedIndex m.1)).unop)
1898
1899 def bouquetValue (X Y : Folio) (m : Fin (bouquetLength X Y)) :
1900   (bouquetChain X Y m).Obj :=
1901   Fin.cases () (fun n => toLex (Sum.inl (X.pageValue (X.paddedIndex n.1)))) m
1902
1903 def bouquetConMap (X Y : Folio) {i j : Fin (bouquetLength X Y)} (h : i <= j) :
1904   bouquetChain X Y j --> bouquetChain X Y i := by
1905   revert j
1906   refine Fin.cases ?_ (fun i => ?_) i
1907   . rename_i j
1908   intro h
1909   refine
1910     { toFun := fun _ => ()
1911       monotone := fun _ _ => le_rfl
1912       point_surjective := fun z => by
1913         refine <|bouquetValue X Y j, ?_|>
1914         change (() : Unit) = z
1915         exact Subsingleton.elim _ _ }
1916   . rename_i j
1917   intro h
1918   revert h
1919   refine Fin.cases (by
1920     intro h
1921     change i.1 + 1 <= 0 at h
1922     omega) (fun j => ?_) j
1923   intro h
1924   have hij : i <= j := Fin.succ_le_succ_iff.mp h
1925   let hx : X.paddedIndex i.1 <= X.paddedIndex j.1 := X.paddedIndex_mono hij
1926   let hy : Y.paddedIndex i.1 <= Y.paddedIndex j.1 := Y.paddedIndex_mono hij
1927   exact ConHom.sum (X.core.map (homOfLE hx)).unop
1928     (Y.core.map (homOfLE hy)).unop
1929
1930 def bouquetFolio (X Y : Folio) : Folio where
1931   length := bouquetLength X Y
1932   positive := bouquetLength_pos X Y
1933   core :=
1934     { obj := fun m => op (bouquetChain X Y m)
1935       map := fun f => (bouquetConMap X Y (leOfHom f)).op
1936       map_id := by
1937         intro m
1938         refine Fin.cases ?_ (fun m => ?_) m
1939         . apply Quiver.Hom.unop_inj
1940         apply ConHom.ext
1941         intro z
1942         change (() : Unit) = ()
1943         rfl
1944       apply Quiver.Hom.unop_inj
1945       apply ConHom.ext
1946       intro w
1947       generalize hz : ofLex w = z
1948       rcases z with z | z
1949       . have hz' : w = toLex (Sum.inl z) := ofLex.injective (by simp using hz)
1950         subst w
1951         simp [bouquetConMap, ConHom.sum]
1952         change toLex (Sum.inl z) = toLex (Sum.inl z)
1953         rfl
1954       . have hz' : w = toLex (Sum.inr z) := ofLex.injective (by simp using hz)
1955         subst w
1956         simp [bouquetConMap, ConHom.sum]
1957         change toLex (Sum.inr z) = toLex (Sum.inr z)
1958         rfl
1959     map_comp := by
1960       intro i j k f g
1961       letI : NeZero (bouquetLength X Y) := <|Nat.ne_of_gt (bouquetLength_pos X Y)|>
1962       revert j k
1963       refine Fin.cases ?_ (fun i => ?_) i
1964       . rename_i jTarget kTarget

```

```

1965     intro f g
1966     apply Quiver.Hom.unop_inj
1967     apply ConHom.ext
1968     intro z
1969     change (()) : Unit = ()
1970     rfl
1971   . rename_i iOrig jTarget kTarget
1972     intro f g
1973     have hj : jTarget != 0 := by
1974       intro e
1975       have hf := leOfHom f
1976       rw [e] at hf
1977       change i.1 + 1 <= 0 at hf
1978       omega
1979     obtain <|j, rfl|> := Fin.eq_succ_of_ne_zero hj
1980     have hk : kTarget != 0 := by
1981       intro e
1982       have hg := leOfHom g
1983       rw [e] at hg
1984       change j.1 + 1 <= 0 at hg
1985       omega
1986     obtain <|k, rfl|> := Fin.eq_succ_of_ne_zero hk
1987     apply Quiver.Hom.unop_inj
1988     apply ConHom.ext
1989     intro w
1990     generalize hz : ofLex w = z
1991     rcases z with z | z
1992     . have hz' : w = toLex (Sum.inl z) := ofLex.injective (by simp using hz)
1993       subst w
1994       simp only [bouquetConMap, ConHom.sum] using congrArg
1995       (fun q => toLex (Sum.inl q)) (X.map_unop_comp_apply _ _ z)
1996     . have hz' : w = toLex (Sum.inr z) := ofLex.injective (by simp using hz)
1997       subst w
1998       simp only [bouquetConMap, ConHom.sum] using congrArg
1999       (fun q => toLex (Sum.inr q)) (Y.map_unop_comp_apply _ _ z) }
2000   originEquiv := by
2001     exact singletonObjEquiv
2002
2003   def domSum (X Y : DomIns) : DomIns where
2004     toDom :=
2005       { Carrier := Sum X Y
2006         rank :=
2007           { toFun := fun z => match z with
2008             | .inl x => 2 * X.toDom.rank x
2009             | .inr y => 2 * Y.toDom.rank y + 1
2010           inj' := by
2011             intro a b h
2012             rcases a with a | a <;> rcases b with b | b
2013             . simp only at h
2014             exact congrArg Sum.inl (X.toDom.rank.injective (by omega))
2015             . simp only at h
2016             omega
2017             . simp only at h
2018             omega
2019             . simp only at h
2020             exact congrArg Sum.inr (Y.toDom.rank.injective (by omega)) } }
2021
2022   def domSum.inl (X Y : DomIns) : X --> domSum X Y where
2023     toFun := Sum.inl
2024     inj' := Sum.inl_injective
2025
2026   def domSum.inr (X Y : DomIns) : Y --> domSum X Y where
2027     toFun := Sum.inr
2028     inj' := Sum.inr_injective
2029
2030   def domSum.map {X_1 X_2 Y_1 Y_2 : DomIns} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2031     domSum X_1 X_2 --> domSum Y_1 Y_2 where
2032     toFun := Sum.map f g
2033     inj' := by
2034       intro a b h
2035       rcases a with a | a <;> rcases b with b | b
2036       . exact congrArg Sum.inl (f.injective (Sum.inl.inj h))
2037       . exact Sum.noConfusion h

```

```

2038     . exact Sum.noConfusion h
2039     . exact congrArg Sum.inr (g.injective (Sum.inr.inj h))
2040
2041 def tallMergePage (X Y : TallAtlas) : Nat -> Type
2042 | 0 => Unit
2043 | n + 1 => Sum (X.H.obj (op n)) (Y.H.obj (op n))
2044
2045 def tallMergePageMap (X Y : TallAtlas) {i j : Nat} (h : j <= i) :
2046   tallMergePage X Y i -> tallMergePage X Y j := by
2047   induction j with
2048   | zero => exact fun _ => ()
2049   | succ j =>
2050     induction i with
2051     | zero => omega
2052     | succ i =>
2053       exact Sum.map
2054         (X.H.map (homOfLE (Nat.succ_le_succ_iff.mp h)).op)
2055         (Y.H.map (homOfLE (Nat.succ_le_succ_iff.mp h)).op)
2056
2057 def tallMergeH (X Y : TallAtlas) : Natop ==> Type where
2058   obj n := tallMergePage X Y n.unop
2059   map f := tallMergePageMap X Y (leOfHom f.unop)
2060   map_id n := by
2061     funext z
2062     rcases n with <|n|>
2063     cases n with
2064     | zero => rfl
2065     | succ n =>
2066       rcases z with z | z
2067       . apply congrArg Sum.inl
2068       simp
2069       . apply congrArg Sum.inr
2070       simp
2071   map_comp := by
2072     intro ni nj nk f g
2073     funext z
2074     rcases ni with <|i|>
2075     rcases nj with <|j|>
2076     rcases nk with <|k|>
2077     cases k with
2078     | zero => rfl
2079     | succ k =>
2080       have hj : j != 0 := by
2081         intro e
2082         subst j
2083         have hg := leOfHom g.unop
2084         change k + 1 <= 0 at hg
2085         omega
2086       obtain <|j, rfl|> := Nat.exists_eq_succ_of_ne_zero hj
2087       have hi : i != 0 := by
2088         intro e
2089         subst i
2090         have hf := leOfHom f.unop
2091         change j + 1 <= 0 at hf
2092         omega
2093       obtain <|i, rfl|> := Nat.exists_eq_succ_of_ne_zero hi
2094       let fij : (op i : Natop) --> op j :=
2095         (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom f.unop))).op
2096       let fjk : (op j : Natop) --> op k :=
2097         (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom g.unop))).op
2098       let fik : (op i : Natop) --> op k :=
2099         (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom (f >> g).unop))).op
2100       have hcomp : fik = fij >> fjk := Subsingleton.elim _ _
2101       rcases z with z | z
2102       . apply congrArg Sum.inl
2103       change X.H.map fik z = X.H.map fjk (X.H.map fij z)
2104       rw [hcomp, X.H.map_comp]
2105       rfl
2106       . apply congrArg Sum.inr
2107       change Y.H.map fik z = Y.H.map fjk (Y.H.map fij z)
2108       rw [hcomp, Y.H.map_comp]
2109       rfl
2110

```

```

2111 def bouquetPag (X Y : Atl) : Pag := <|bouquetFolio X.P.folio Y.P.folio|>
2112
2113 def bouquetLeftIndex (X Y : Atl) (m : Fin X.P.folio.length) :
2114   Fin (bouquetLength X.P.folio Y.P.folio) :=
2115   <|m.1 + 1, by simp [bouquetLength]|>
2116
2117 def bouquetRightIndex (X Y : Atl) (m : Fin Y.P.folio.length) :
2118   Fin (bouquetLength X.P.folio Y.P.folio) :=
2119   <|m.1 + 1, by simp [bouquetLength]|>
2120
2121 def bouquetLeftPred (X Y : Atl) (m : Fin X.P.folio.length) :
2122   Fin (bouquetDepth X.P.folio Y.P.folio) :=
2123   <|m.1, lt_of_lt_of_le m.2 (le_max_left _ _)|>
2124
2125 def bouquetRightPred (X Y : Atl) (m : Fin Y.P.folio.length) :
2126   Fin (bouquetDepth X.P.folio Y.P.folio) :=
2127   <|m.1, lt_of_lt_of_le m.2 (le_max_right _ _)|>
2128
2129 @[simp]
2130 theorem bouquetLeftIndex_eq_succ (X Y : Atl) (m : Fin X.P.folio.length) :
2131   bouquetLeftIndex X Y m = (bouquetLeftPred X Y m).succ := rfl
2132
2133 @[simp]
2134 theorem bouquetRightIndex_eq_succ (X Y : Atl) (m : Fin Y.P.folio.length) :
2135   bouquetRightIndex X Y m = (bouquetRightPred X Y m).succ := rfl
2136
2137 def bouquetLeftElement (X Y : Atl) (x : X.E) : (bouquetPag X Y).E := by
2138   refine <|op (bouquetLeftPred X Y x.1.unop).succ, ?_||>
2139   change (bouquetChain X.P.folio Y.P.folio
2140     (bouquetLeftPred X Y x.1.unop).succ).Obj
2141   change Lex (Sum
2142     ((X.P.folio.core.obj (X.P.folio.paddedIndex x.1.unop.1)).unop.Obj)
2143     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex x.1.unop.1)).unop.Obj))
2144   exact toLex (Sum.inl (X.P.H.map
2145     (eqToHom (congrArg op (X.P.folio.paddedIndex_fin x.1.unop).symm)) x.2))
2146
2147 def bouquetRightElement (X Y : Atl) (y : Y.E) : (bouquetPag X Y).E := by
2148   refine <|op (bouquetRightPred X Y y.1.unop).succ, ?_||>
2149   change (bouquetChain X.P.folio Y.P.folio
2150     (bouquetRightPred X Y y.1.unop).succ).Obj
2151   change Lex (Sum
2152     ((X.P.folio.core.obj (X.P.folio.paddedIndex y.1.unop.1)).unop.Obj)
2153     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex y.1.unop.1)).unop.Obj))
2154   exact toLex (Sum.inr (Y.P.H.map
2155     (eqToHom (congrArg op (Y.P.folio.paddedIndex_fin y.1.unop).symm)) y.2))
2156
2157 def bouquetLeftMapHom (X Y : Atl) {x y : X.E} (q : x --> y) :
2158   bouquetLeftElement X Y x --> bouquetLeftElement X Y y := by
2159   rcases x with <|<|mx|>, kx|>
2160   rcases y with <|<|my|>, ky|>
2161   let hxy : my <= mx := leOfHom (Quiver.Hom.unop q.val)
2162   let h : bouquetLeftPred X Y my <= bouquetLeftPred X Y mx := by
2163     change my.1 <= mx.1
2164     exact hxy
2165   refine CategoryOfElements.homMk _ _
2166     (homOfLE (Fin.succ_le_succ_iff.mpr h)).op ?_
2167   dsimp only [bouquetLeftElement]
2168   simp only [bouquetLeftPred, bouquetPag, bouquetFolio, Folio.H,
2169     bouquetChain, bouquetConMap, Fin.cases_succ, ConHom.sum, Functor.leftOp_map,
2170     Functor.comp_obj, Functor.comp_map, Quiver.Hom.unop_op, Tra]
2171   change toLex (Sum.inl _) = toLex (Sum.inl _)
2172   congr 2
2173   have hqv : X.P.H.map q.val kx = ky := q.property
2174   rw [<- hqv]
2175   let hp : X.P.folio.paddedIndex my.1 <= X.P.folio.paddedIndex mx.1 :=
2176     X.P.folio.paddedIndex_mono hxy
2177   change X.P.H.map (homOfLE hp).op
2178     (X.P.H.map (eqToHom _) kx) = X.P.H.map (eqToHom _)
2179     (X.P.H.map q.val kx)
2180   rw [<- FunctorToTypes.map_comp_apply]
2181   rw [<- FunctorToTypes.map_comp_apply]
2182   congr 1
2183

```

```

2184 def bouquetRightMapHom (X Y : At1) {x y : Y.E} (q : x --> y) :
2185   bouquetRightElement X Y x --> bouquetRightElement X Y y := by
2186   rcases x with <|<|mx|>, kx|>
2187   rcases y with <|<|my|>, ky|>
2188   let hxy : my <= mx := leOfHom (Quiver.Hom.unop q.val)
2189   let h : bouquetRightPred X Y my <= bouquetRightPred X Y mx := by
2190     change my.1 <= mx.1
2191     exact hxy
2192   refine CategoryOfElements.homMk _ _
2193     (homOfLE (Fin.succ_le_succ_iff.mpr h)).op ?_
2194   dsimp only [bouquetRightElement]
2195   simp only [bouquetRightPred, bouquetPag, bouquetFolio, Folio.H,
2196     bouquetChain, bouquetConMap, Fin.cases_succ, ConHom.sum, Functor.leftOp_map,
2197     Functor.comp_obj, Functor.comp_map, Quiver.Hom.unop_op, Tra]
2198   change toLex (Sum.inr _) = toLex (Sum.inr _)
2199   congr 2
2200   have hqv : Y.P.H.map q.val kx = ky := q.property
2201   rw [<- hqv]
2202   let hp : Y.P.folio.paddedIndex my.1 <= Y.P.folio.paddedIndex mx.1 :=
2203     Y.P.folio.paddedIndex_mono hxy
2204   change Y.P.H.map (homOfLE hp).op
2205     (Y.P.H.map (eqToHom _) kx) = Y.P.H.map (eqToHom _)
2206     (Y.P.H.map q.val kx)
2207   rw [<- FunctorToTypes.map_comp_apply]
2208   rw [<- FunctorToTypes.map_comp_apply]
2209   congr 1
2210
2211 def bouquetLeftElements (X Y : At1) : X.E ==> (bouquetPag X Y).E where
2212   obj := bouquetLeftElement X Y
2213   map := bouquetLeftMapHom X Y
2214   map_id _ := by
2215     apply CategoryOfElements.ext
2216     apply Subsingleton.elim
2217   map_comp _ _ := by
2218     apply CategoryOfElements.ext
2219     apply Subsingleton.elim
2220
2221 def bouquetRightElements (X Y : At1) : Y.E ==> (bouquetPag X Y).E where
2222   obj := bouquetRightElement X Y
2223   map := bouquetRightMapHom X Y
2224   map_id _ := by
2225     apply CategoryOfElements.ext
2226     apply Subsingleton.elim
2227   map_comp _ _ := by
2228     apply CategoryOfElements.ext
2229     apply Subsingleton.elim
2230
2231 def imageDom {A R : DomIns} (f : A --> R) : DomIns where
2232   toDom :=
2233     { Carrier := Set.range f
2234       rank :=
2235         ({ toFun := Subtype.val
2236           inj' := Subtype.val_injective } : Set.range f --> R).trans R.toDom.rank }
2237
2238 def imageDom.inclusion {A R : DomIns} (f : A --> R) : imageDom f --> R where
2239   toFun := Subtype.val
2240   inj' := Subtype.val_injective
2241
2242 def imageDom.map {A B R : DomIns} (f : A --> R) (g : B --> R)
2243   (h : Set.range f subset Set.range g) : imageDom f --> imageDom g where
2244   toFun z := <|z.1, h z.2|>
2245   inj' := by
2246     intro a b e
2247     apply Subtype.ext
2248     exact congrArg (fun q => q.1) e
2249
2250 def imageDom.mapAcross {A B R S : DomIns} (f : A --> R) (g : B --> S)
2251   (h : R --> S) (factor : forall a, exists b, h (f a) = g b) :
2252   imageDom f --> imageDom g where
2253   toFun z := <|h z.1, by
2254     rcases z.2 with <|a, ha|>
2255     rcases factor a with <|b, hb|>
2256     exact <|b, (congrArg h ha.symm |>.trans hb).symm|>|>

```

```

2257   inj' := by
2258     intro a b e
2259     apply Subtype.ext
2260     apply h.injective
2261     exact congrArg Subtype.val e
2262
2263   def tallMergeRawDom (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) : DomIns := by
2264     rcases x with <|<|n|>, v|>
2265     cases n with
2266     | zero => exact domSum X.extent Y.extent
2267     | succ n =>
2268       exact match v with
2269       | .inl k => X.G.obj <|op n, k|>
2270       | .inr k => Y.G.obj <|op n, k|>
2271
2272   def tallMergeRootMap (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) :
2273     tallMergeRawDom X Y x --> domSum X.extent Y.extent := by
2274     rcases x with <|<|n|>, v|>
2275     cases n with
2276     | zero => exact Id _
2277     | succ n =>
2278       exact match v with
2279       | .inl k => X.G.map (X.toOrigin <|op n, k|>) >>
2280         domSum.inl X.extent Y.extent
2281       | .inr k => Y.G.map (Y.toOrigin <|op n, k|>) >>
2282         domSum.inr X.extent Y.extent
2283
2284   def tallMergeDom (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) : DomIns :=
2285     imageDom (tallMergeRootMap X Y x)
2286
2287   theorem tallMergeRange_mono (X Y : TallAtlas)
2288     {x y : (tallMergeH X Y).Elements} (f : x --> y) :
2289     Set.range (tallMergeRootMap X Y x) subset
2290     Set.range (tallMergeRootMap X Y y) := by
2291     intro z hz
2292     rcases x with <|<|mx|>, vx|>
2293     rcases y with <|<|my|>, vy|>
2294     rcases hz with <|a, rfl|>
2295     cases mx with
2296     | zero =>
2297       have hmy : my = 0 := by
2298         have hf := leOfHom f.val.unop
2299         change my <= 0 at hf
2300         omega
2301       subst my
2302       exact <|a, rfl|>
2303     | succ n =>
2304       cases my with
2305       | zero => exact <|tallMergeRootMap X Y <|op n.succ, vx|> a, rfl|>
2306       | succ p =>
2307         have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom f.val.unop)
2308         rcases vx with k | k
2309         · have hfv : Sum.inl (X.H.map (homOfLE hpn).op k) = vy := by
2310             simp [tallMergeH, tallMergePageMap] using f.property
2311             subst vy
2312             let q := CategoryOfElements.homMk
2313               (<|op n, k|> : X.E)
2314               (<|op p, X.H.map (homOfLE hpn).op k|> : X.E)
2315               (homOfLE hpn).op rfl
2316             refine <|X.G.map q a, ?_|>
2317             change Sum.inl (X.G.map (X.toOrigin _) (X.G.map q a)) =
2318               Sum.inl (X.G.map (X.toOrigin _) a)
2319             apply congrArg Sum.inl
2320             have hc : q >> X.toOrigin _ = X.toOrigin _ :=
2321               CategoryOfElements.ext X.H _ (Subsingleton.elim _ _)
2322             have hm := X.G.map_comp q (X.toOrigin _)
2323             rw [hc] at hm
2324             exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
2325         · have hfv : Sum.inr (Y.H.map (homOfLE hpn).op k) = vy := by
2326             simp [tallMergeH, tallMergePageMap] using f.property
2327             subst vy
2328             let q := CategoryOfElements.homMk
2329               (<|op n, k|> : Y.E)

```

```

2330         (<|op p, Y.H.map (homOfLE hpn).op k|> : Y.E)
2331         (homOfLE hpn).op rfl
2332     refine <|Y.G.map q a, ?_|>
2333     change Sum.inr (Y.G.map (Y.toOrigin _) (Y.G.map q a)) =
2334         Sum.inr (Y.G.map (Y.toOrigin _) a)
2335     apply congrArg Sum.inr
2336     have hc : q >> Y.toOrigin _ = Y.toOrigin _ :=
2337         CategoryOfElements.ext Y.H _ _ (Subsingleton.elim _ _)
2338     have hm := Y.G.map_comp q (Y.toOrigin _)
2339     rw [hc] at hm
2340     exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
2341
2342 def tallMergeImageMap (X Y : TallAtlas)
2343   {x y : (tallMergeH X Y).Elements} (f : x --> y) :
2344   tallMergeDom X Y x --> tallMergeDom X Y y :=
2345   imageDom.map _ _ (tallMergeRange_mono X Y f)
2346
2347 def tallMergeG (X Y : TallAtlas) : (tallMergeH X Y).Elements ==> DomIns where
2348   obj := tallMergeDom X Y
2349   map := tallMergeImageMap X Y
2350   map_id _ := by
2351     apply DomIns.hom_ext
2352     intro z
2353     apply Subtype.ext
2354     rfl
2355   map_comp _ _ := by
2356     apply DomIns.hom_ext
2357     intro z
2358     apply Subtype.ext
2359     rfl
2360
2361 def tallMergeCellLT (X Y : TallAtlas)
2362   (x y : (tallMergeH X Y).Elements) : Prop := by
2363   rcases x with <|<|nx|>, vx|>
2364   rcases y with <|<|ny|>, vy|>
2365   cases nx with
2366   | zero => exact False
2367   | succ nx =>
2368     cases ny with
2369     | zero => exact False
2370     | succ ny =>
2371       exact match vx, vy with
2372       | .inl k, .inl l => X.cellLT <|op nx, k|> <|op ny, l|>
2373       | .inl _, .inr _ => True
2374       | .inr _, .inl _ => False
2375       | .inr k, .inr l => Y.cellLT <|op nx, k|> <|op ny, l|>
2376
2377 def tallMerge (X Y : TallAtlas) : TallAtlas where
2378   H := tallMergeH X Y
2379   originValue := ()
2380   originUnique x := by
2381     change Unit at x
2382     cases x
2383     rfl
2384   G := tallMergeG X Y
2385   cellLT := tallMergeCellLT X Y
2386   coveredExtent t := match t.1 with
2387   | .inl a => X.coveredExtent a
2388   | .inr b => Y.coveredExtent b
2389
2390 def tallMergeLeftObj (X Y : TallAtlas) (x : X.E) : (tallMerge X Y).E :=
2391   <|op (x.1.unop + 1), Sum.inl x.2|>
2392
2393 def tallMergeRightObj (X Y : TallAtlas) (y : Y.E) : (tallMerge X Y).E :=
2394   <|op (y.1.unop + 1), Sum.inr y.2|>
2395
2396 def tallMergeLeftMap (X Y : TallAtlas) {x y : X.E} (f : x --> y) :
2397   tallMergeLeftObj X Y x --> tallMergeLeftObj X Y y := by
2398   rcases x with <|<|n|>, kx|>
2399   rcases y with <|<|m|>, ky|>
2400   have hmn : m <= n := leOfHom f.val.unop
2401   refine CategoryOfElements.homMk _ _
2402   (homOfLE (Nat.succ_le_succ hmn)).op ?_

```

```

2403   change Sum.inl (X.H.map (homOfLE hmn).op kx) = Sum.inl ky
2404   congr 1
2405   have ef : f.val = (homOfLE hmn).op := Subsingleton.elim _ _
2406   rw [← ef]
2407   exact f.property
2408
2409   def tallMergeRightMap (X Y : TallAtlas) {x y : Y.E} (f : x --> y) :
2410     tallMergeRightObj X Y x --> tallMergeRightObj X Y y := by
2411     rcases x with <|<n|>, kx|>
2412     rcases y with <|<m|>, ky|>
2413     have hmn : m <= n := leOfHom f.val.unop
2414     refine CategoryOfElements.homMk _ _
2415       (homOfLE (Nat.succ_le_succ hmn)).op ?_
2416     change Sum.inr (Y.H.map (homOfLE hmn).op kx) = Sum.inr ky
2417     congr 1
2418     have ef : f.val = (homOfLE hmn).op := Subsingleton.elim _ _
2419     rw [← ef]
2420     exact f.property
2421
2422   def tallMergeLeft (X Y : TallAtlas) : X.E ==> (tallMerge X Y).E where
2423     obj := tallMergeLeftObj X Y
2424     map := tallMergeLeftMap X Y
2425     map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2426     map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2427
2428   def tallMergeRight (X Y : TallAtlas) : Y.E ==> (tallMerge X Y).E where
2429     obj := tallMergeRightObj X Y
2430     map := tallMergeRightMap X Y
2431     map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2432     map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2433
2434   def tallHorPObj {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2)
2435     (x : (tallMerge X_1 X_2).E) : (tallMerge Y_1 Y_2).E := by
2436     rcases x with <|<n|>, v|>
2437     cases n with
2438     | zero => exact (tallMerge Y_1 Y_2).originElement
2439     | succ n =>
2440       exact match v with
2441       | .inl k => tallMergeLeftObj Y_1 Y_2 (f.P.obj <|op n, k|>)
2442       | .inr k => tallMergeRightObj Y_1 Y_2 (g.P.obj <|op n, k|>)
2443
2444   def tallHorPHom {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2)
2445     {x y : (tallMerge X_1 X_2).E} (q : x --> y) :
2446     tallHorPObj f g x --> tallHorPObj f g y := by
2447     rcases x with <|<mx|>, vx|>
2448     rcases y with <|<my|>, vy|>
2449     cases mx with
2450     | zero =>
2451       have hmy : my = 0 := by
2452       have hq := leOfHom q.val.unop
2453       change my <= 0 at hq
2454       omega
2455       subst my
2456       exact Id _
2457     | succ n =>
2458       cases my with
2459       | zero => exact (tallMerge Y_1 Y_2).toOrigin _
2460       | succ p =>
2461         have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom q.val.unop)
2462         rcases vx with k | k
2463         . have hqv : Sum.inl (X_1.H.map (homOfLE hpn).op k) = vy := by
2464           simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
2465           subst vy
2466           let r := CategoryOfElements.homMk
2467             (<|op n, k|> : X_1.E)
2468             (<|op p, X_1.H.map (homOfLE hpn).op k|> : X_1.E)
2469             (homOfLE hpn).op rfl
2470           exact tallMergeLeftMap Y_1 Y_2 (f.P.map r)
2471         . have hqv : Sum.inr (X_2.H.map (homOfLE hpn).op k) = vy := by
2472           simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
2473           subst vy
2474           let r := CategoryOfElements.homMk
2475             (<|op n, k|> : X_2.E)

```



```

2476         (⟦op p, X_2.H.map (homOfLE hpn).op k⟧ : X_2.E)
2477         (homOfLE hpn).op rfl
2478     exact tallMergeRightMap Y_1 Y_2 (g.P.map r)
2479
2480 def tallHorP {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2481   (tallMerge X_1 X_2).E ==> (tallMerge Y_1 Y_2).E where
2482   obj := tallHorPObj f g
2483   map := tallHorPHom f g
2484   map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2485   map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2486
2487 def TallAtlas.originImage (X : TallAtlas) (x : X.E) (a : X.G.obj x) : X.extent :=
2488   X.G.map (X.toOrigin x) a
2489
2490 @[simp]
2491 theorem TallAtlas.originImage_origin (X : TallAtlas) (a : X.extent) :
2492   X.originImage X.originElement a = a := by
2493   change X.G.map (X.toOrigin X.originElement) a = a
2494   have h : X.toOrigin X.originElement = Id X.originElement :=
2495     Subsingleton.elim _ _
2496   rw [h, X.G.map_id]
2497   rfl
2498
2499 def TallAtlas.extentMap {X Y : TallAtlas} (f : X --> Y) : X.extent --> Y.extent :=
2500   f.A.app X.originElement >> Y.G.map (Y.toOrigin (f.P.obj X.originElement))
2501
2502 @[simp]
2503 theorem TallAtlas.extentMap_id (X : TallAtlas) :
2504   TallAtlas.extentMap (Id X) = Id X.extent := by
2505   apply DomIns.hom_ext
2506   intro a
2507   change X.G.map (X.toOrigin X.originElement) a = a
2508   have h : X.toOrigin X.originElement = Id X.originElement := Subsingleton.elim _ _
2509   rw [h]
2510   rw [X.G.map_id]
2511   rfl
2512
2513 theorem TallAtlas.extentMap_originImage {X Y : TallAtlas} (f : X --> Y)
2514   (x : X.E) (a : X.G.obj x) :
2515   TallAtlas.extentMap f (X.originImage x a) =
2516     Y.originImage (f.P.obj x) (f.A.app x a) := by
2517   simp only [TallAtlas.extentMap, TallAtlas.originImage]
2518   have hn := f.A.naturality (X.toOrigin x)
2519   have hc : f.P.map (X.toOrigin x) >> Y.toOrigin (f.P.obj X.originElement) =
2520     Y.toOrigin (f.P.obj x) := by
2521     apply CategoryOfElements.ext
2522     apply Subsingleton.elim
2523   rw [← hc, Y.G.map_comp]
2524   exact congrFun (congrArg Function.Embedding.toFun
2525     (congrArg (fun k => k >> Y.G.map (Y.toOrigin (f.P.obj X.originElement)))) hn) a
2526
2527 @[simp]
2528 theorem TallAtlas.extentMap_comp {X Y Z : TallAtlas} (f : X --> Y) (g : Y --> Z) :
2529   TallAtlas.extentMap (f >> g) =
2530     TallAtlas.extentMap f >> TallAtlas.extentMap g := by
2531   simp only [TallAtlas.extentMap]
2532   have hn := g.A.naturality (Y.toOrigin (f.P.obj X.originElement))
2533   simp only [Functor.comp_map] at hn
2534   have hc : g.P.map (Y.toOrigin (f.P.obj X.originElement)) >>
2535     Z.toOrigin (g.P.obj Y.originElement) =
2536     Z.toOrigin (g.P.obj (f.P.obj X.originElement)) := by
2537     apply CategoryOfElements.ext
2538     apply Subsingleton.elim
2539   change f.A.app X.originElement >> g.A.app (f.P.obj X.originElement) >>
2540     Z.G.map (Z.toOrigin (g.P.obj (f.P.obj X.originElement))) =
2541     (f.A.app X.originElement >> Y.G.map (Y.toOrigin (f.P.obj X.originElement))) >>
2542     g.A.app Y.originElement >> Z.G.map (Z.toOrigin (g.P.obj Y.originElement))
2543   simp only [Category.assoc]
2544   rw [← Category.assoc (Y.G.map (Y.toOrigin (f.P.obj X.originElement)))]
2545     (g.A.app Y.originElement) (Z.G.map (Z.toOrigin (g.P.obj Y.originElement)))
2546   rw [hn]
2547   rw [Category.assoc (g.A.app (f.P.obj X.originElement))]
2548   rw [← Z.G.map_comp]

```

```

2549   rw [hc]
2550
2551   def tallHorExtentMap {X_1 X_2 Y_1 Y_2 : TallAtlas}
2552     (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2553     domSum X_1.extent X_2.extent --> domSum Y_1.extent Y_2.extent :=
2554     domSum.map (TallAtlas.extentMap f) (TallAtlas.extentMap g)
2555
2556   theorem tallHorRootFactor {X_1 X_2 Y_1 Y_2 : TallAtlas}
2557     (f : X_1 --> Y_1) (g : X_2 --> Y_2)
2558     (x : (tallMerge X_1 X_2).E) (a : tallMergeRawDom X_1 X_2 x) :
2559     exists b : tallMergeRawDom Y_1 Y_2 (tallHorPObj f g x),
2560     tallHorExtentMap f g (tallMergeRootMap X_1 X_2 x a) =
2561     tallMergeRootMap Y_1 Y_2 (tallHorPObj f g x) b := by
2562   rcases x with <|<n|>, v|>
2563   cases n with
2564   | zero => exact <|tallHorExtentMap f g a, rfl|>
2565   | succ n =>
2566     rcases v with k | k
2567     . let x_0 : X_1.E := <|op n, k|>
2568     . let y_0 := f.P.obj x_0
2569     . refine <|f.A.app x_0 a, ?_|>
2570     . change Sum.inl (TallAtlas.extentMap f (X_1.originImage x_0 a)) =
2571     . Sum.inl (Y_1.originImage y_0 (f.A.app x_0 a))
2572     . exact congrArg Sum.inl (TallAtlas.extentMap_originImage f x_0 a)
2573     . let x_0 : X_2.E := <|op n, k|>
2574     . let y_0 := g.P.obj x_0
2575     . refine <|g.A.app x_0 a, ?_|>
2576     . change Sum.inr (TallAtlas.extentMap g (X_2.originImage x_0 a)) =
2577     . Sum.inr (Y_2.originImage y_0 (g.A.app x_0 a))
2578     . exact congrArg Sum.inr (TallAtlas.extentMap_originImage g x_0 a)
2579
2580   def tallHorA {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2581     (tallMerge X_1 X_2).G --> tallHorP f g then (tallMerge Y_1 Y_2).G where
2582     app x := imageDom.mapAcross
2583       (tallMergeRootMap X_1 X_2 x)
2584       (tallMergeRootMap Y_1 Y_2 (tallHorPObj f g x))
2585     (tallHorExtentMap f g) (tallHorRootFactor f g x)
2586     naturality := by
2587     intro x y q
2588     apply DomIns.hom_ext
2589     intro z
2590     apply Subtype.ext
2591     rfl
2592
2593   def tallHorMap {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2594     tallMerge X_1 X_2 --> tallMerge Y_1 Y_2 where
2595     P := tallHorP f g
2596     A := tallHorA f g
2597
2598   @[simp]
2599   theorem tallHorMap_extentMap_val {X_1 X_2 Y_1 Y_2 : TallAtlas}
2600     (f : X_1 --> Y_1) (g : X_2 --> Y_2)
2601     (z : (tallMerge X_1 X_2).extent) :
2602     (TallAtlas.extentMap (tallHorMap f g) z).1 =
2603     tallHorExtentMap f g z.1 := by
2604     rfl
2605
2606   theorem tallHorMap_id (X Y : TallAtlas) :
2607     tallHorMap (Id X) (Id Y) = Id (tallMerge X Y) := by
2608     have hP : tallHorP (Id X) (Id Y) = Unit (tallMerge X Y).E := by
2609     exact CategoryTheory.Functor.ext (F := tallHorP (Id X) (Id Y))
2610       (G := Unit (tallMerge X Y).E) (fun x => by
2611       rcases x with <|<n|>, v|>
2612       cases n with
2613       | zero => rfl
2614       | succ n => cases v <|> rfl)
2615     have hA : HEq (tallHorMap (Id X) (Id Y)).A
2616       (Id (tallMerge X Y) : TallAtlasHom _ _).A := by
2617     apply NatTrans.hext_right _ _
2618     (congrArg (fun P => P then (tallMerge X Y).G) hP)
2619     intro x
2620     rcases x with <|<n|>, v|>
2621     cases n with

```

```

2622 | zero =>
2623   apply heq_of_eq
2624   apply DomIns.hom_ext
2625   intro z
2626   apply Subtype.ext
2627   simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap]
2628   rw [TallAtlas.extentMap_id, TallAtlas.extentMap_id]
2629   change Sum.map (Id X.extent) (Id Y.extent) z.1 = z.1
2630   rcases z.1 with a | b <|> rfl
2631 | succ n =>
2632   rcases v with v | v <|>
2633   apply heq_of_eq <|>
2634   apply DomIns.hom_ext <|>
2635   intro z <|>
2636   apply Subtype.ext <|>
2637   simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap] <|>
2638   rw [TallAtlas.extentMap_id, TallAtlas.extentMap_id] <|>
2639   change Sum.map (Id X.extent) (Id Y.extent) z.1 = z.1 <|>
2640   rcases z.1 with a | b <|> rfl
2641 exact TallAtlasHom.ext _ _ hP hA
2642
2643 theorem tallHorMap_comp {X_1 X_2 Y_1 Y_2 Z_1 Z_2 : TallAtlas}
2644   (f_1 : X_1 --> Y_1) (f_2 : Y_1 --> Z_1) (g_1 : X_2 --> Y_2) (g_2 : Y_2 --> Z_2) :
2645   tallHorMap (f_1 >> f_2) (g_1 >> g_2) =
2646   tallHorMap f_1 g_1 >> tallHorMap f_2 g_2 := by
2647   have hP : tallHorP (f_1 >> f_2) (g_1 >> g_2) =
2648   tallHorP f_1 g_1 then tallHorP f_2 g_2 := by
2649   exact CategoryTheory.Functor.ext
2650   (F := tallHorP (f_1 >> f_2) (g_1 >> g_2))
2651   (G := tallHorP f_1 g_1 then tallHorP f_2 g_2) (fun x => by
2652     rcases x with <|<|n|>, v|>
2653     cases n with
2654     | zero => rfl
2655     | succ n => cases v <|> rfl)
2656   have hA : HEq (tallHorMap (f_1 >> f_2) (g_1 >> g_2)).A
2657   (tallHorMap f_1 g_1 >> tallHorMap f_2 g_2).A := by
2658   apply NatTrans.hext_right _ _
2659   (congrArg (fun P => P then (tallMerge Z_1 Z_2).G) hP)
2660   intro x
2661   rcases x with <|<|n|>, v|>
2662   cases n with
2663   | zero =>
2664     apply heq_of_eq
2665     apply DomIns.hom_ext
2666     intro z
2667     apply Subtype.ext
2668     simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap,
2669     TallAtlas.extentMap_comp]
2670     change (domSum.map
2671     (TallAtlas.extentMap f_1 >> TallAtlas.extentMap f_2)
2672     (TallAtlas.extentMap g_1 >> TallAtlas.extentMap g_2)) z.1 =
2673     domSum.map (TallAtlas.extentMap f_2) (TallAtlas.extentMap g_2)
2674     (domSum.map (TallAtlas.extentMap f_1) (TallAtlas.extentMap g_1) z.1)
2675     rcases z.1 with a | b <|> rfl
2676   | succ n =>
2677     rcases v with v | v <|>
2678     apply heq_of_eq <|>
2679     apply DomIns.hom_ext <|>
2680     intro z <|>
2681     apply Subtype.ext <|>
2682     simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap,
2683     TallAtlas.extentMap_comp] <|>
2684     change (domSum.map
2685     (TallAtlas.extentMap f_1 >> TallAtlas.extentMap f_2)
2686     (TallAtlas.extentMap g_1 >> TallAtlas.extentMap g_2)) z.1 =
2687     domSum.map (TallAtlas.extentMap f_2) (TallAtlas.extentMap g_2)
2688     (domSum.map (TallAtlas.extentMap f_1) (TallAtlas.extentMap g_1) z.1) <|>
2689     rcases z.1 with a | b <|> rfl
2690   exact TallAtlasHom.ext _ _ hP hA
2691
2692 /-- The horizontal sum on raw infinite-spine presentations. This is the
2693 implementation bifunctor from which the coherent atlas operation is derived. -/
2694 def TallAtlHorSum : TallAtlas * TallAtlas ==> TallAtlas where

```

```

2695 obj X := tallMerge X.1 X.2
2696 map f := tallHorMap f.1 f.2
2697 map_id X := tallHorMap_id X.1 X.2
2698 map_comp f g := tallHorMap_comp f.1 g.1 f.2 g.2
2699
2700 def domSum.swap (X Y : DomIns) : domSum X Y --> domSum Y X where
2701   toFun z := z.swap
2702   inj' := by
2703     intro a b h
2704     rcases a with a | a <|> rcases b with b | b
2705     . exact congrArg Sum.inl (Sum.inr.inj h)
2706     . cases h
2707     . cases h
2708     . exact congrArg Sum.inr (Sum.inl.inj h)
2709
2710 def tallSwapPObj (X Y : TallAtlas) (x : (tallMerge X Y).E) :
2711   (tallMerge Y X).E := by
2712     rcases x with <|<|n|>, v|>
2713     cases n with
2714     | zero => exact (tallMerge Y X).originElement
2715     | succ n =>
2716       exact match v with
2717       | .inl k => tallMergeRightObj Y X <|op n, k|>
2718       | .inr k => tallMergeLeftObj Y X <|op n, k|>
2719
2720 def tallSwapPHom (X Y : TallAtlas) {x y : (tallMerge X Y).E} (q : x --> y) :
2721   tallSwapPObj X Y x --> tallSwapPObj X Y y := by
2722     rcases x with <|<|mx|>, vx|>
2723     rcases y with <|<|my|>, vy|>
2724     cases mx with
2725     | zero =>
2726       have hmy : my = 0 := by
2727         have hq := leOfHom q.val.unop
2728         change my <= 0 at hq
2729         omega
2730       subst my
2731       exact Id _
2732     | succ n =>
2733       cases my with
2734       | zero => exact (tallMerge Y X).toOrigin _
2735       | succ p =>
2736         have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom q.val.unop)
2737         rcases vx with k | k
2738         . have hqv : Sum.inl (X.H.map (homOfLE hpn).op k) = vy := by
2739           simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
2740           subst vy
2741           exact tallMergeRightMap Y X (CategoryOfElements.homMk _ _
2742             (homOfLE hpn).op rfl)
2743         . have hqv : Sum.inr (Y.H.map (homOfLE hpn).op k) = vy := by
2744           simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
2745           subst vy
2746           exact tallMergeLeftMap Y X (CategoryOfElements.homMk _ _
2747             (homOfLE hpn).op rfl)
2748
2749 def tallSwapP (X Y : TallAtlas) : (tallMerge X Y).E ==> (tallMerge Y X).E where
2750   obj := tallSwapPObj X Y
2751   map := tallSwapPHom X Y
2752   map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2753   map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2754
2755 theorem tallSwapRootFactor (X Y : TallAtlas) (x : (tallMerge X Y).E)
2756   (a : tallMergeRawDom X Y x) :
2757   exists b : tallMergeRawDom Y X (tallSwapPObj X Y x),
2758     domSum.swap X.extent Y.extent (tallMergeRootMap X Y x a) =
2759     tallMergeRootMap Y X (tallSwapPObj X Y x) b := by
2760     rcases x with <|<|n|>, v|>
2761     cases n with
2762     | zero => exact <|a.swap, rfl|>
2763     | succ n =>
2764       rcases v with k | k
2765       . exact <|a, rfl|>
2766       . exact <|a, rfl|>
2767

```

```

2768 def TallAt1Brd (X Y : TallAtlas) : tallMerge X Y --> tallMerge Y X where
2769   P := tallSwapP X Y
2770   A :=
2771     { app := fun x => imageDom.mapAcross
2772       (tallMergeRootMap X Y x)
2773       (tallMergeRootMap Y X (tallSwapPObj X Y x))
2774       (domSum.swap X.extent Y.extent) (tallSwapRootFactor X Y x)
2775     naturality := by
2776       intro x y q
2777       apply DomIns.hom_ext
2778       intro z
2779       apply Subtype.ext
2780       rfl }
2781
2782 theorem TallAt1Brd_involutive (X Y : TallAtlas) :
2783   TallAt1Brd X Y >> TallAt1Brd Y X = Id (tallMerge X Y) := by
2784   have hP : tallSwapP X Y then tallSwapP Y X = Unit (tallMerge X Y).E := by
2785     exact CategoryTheory.Functor.ext (fun x => by
2786       rcases x with <|<n|>, v|>
2787       cases n with
2788       | zero => rfl
2789       | succ n => cases v <|> rfl)
2790   apply TallAtlasHom.ext _ _ hP
2791   apply NatTrans.hext_right _ _
2792   (congrArg (fun P => P then (tallMerge X Y).G) hP)
2793   intro x
2794   rcases x with <|<n|>, v|>
2795   cases n with
2796   | zero =>
2797     apply heq_of_eq
2798     apply DomIns.hom_ext
2799     intro z
2800     apply Subtype.ext
2801     simp only [TallAt1Brd, imageDom.mapAcross]
2802     change domSum.swap Y.extent X.extent
2803     (domSum.swap X.extent Y.extent z.1) = z.1
2804     rcases z.1 with a | b <|> rfl
2805   | succ n =>
2806     rcases v with v | v <|>
2807     apply heq_of_eq <|>
2808     apply DomIns.hom_ext <|>
2809     intro z <|>
2810     apply Subtype.ext <|>
2811     simp only [TallAt1Brd, imageDom.mapAcross] <|>
2812     change domSum.swap Y.extent X.extent
2813     (domSum.swap X.extent Y.extent z.1) = z.1 <|>
2814     rcases z.1 with a | b <|> rfl
2815
2816 def TallAt1BrdIso (X Y : TallAtlas) : tallMerge X Y Iso tallMerge Y X where
2817   hom := TallAt1Brd X Y
2818   inv := TallAt1Brd Y X
2819   hom_inv_id := TallAt1Brd_involutive X Y
2820   inv_hom_id := TallAt1Brd_involutive Y X
2821
2822 /-- The empty atlas regarded on its full spine. -/
2823 def TallAtlasI : TallAtlas := AtlI.tall
2824
2825 theorem domIns_eqToHom_apply {A B : DomIns} (h : A = B) (a : A) :
2826   (eqToHom h : A --> B) a = h transport a := by
2827   cases h
2828   rfl
2829
2830 theorem castDep_symm_cast {A : Sort u} (P : A -> Sort*) {x y : A}
2831   (h : x = y) (a : P y) : h transport (h.symm transport a) = a := by
2832   cases h
2833   rfl
2834
2835 def bouquetRawDomIndex (X Y : Atl) :
2836   (m : Fin (bouquetLength X.P.folio Y.P.folio)) ->
2837   (bouquetFolio X.P.folio Y.P.folio).H.obj (op m) -> DomIns :=
2838   Fin.cases (fun _ => domSum (extent X) (extent Y)) (fun n v =>
2839     match ofLex v with
2840     | .inl k => X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)

```

```

2841 | .inr k => Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k))
2842
2843 theorem bouquetLeftRawDom_eq (X Y : Atl) (x : X.E) :
2844   bouquetRawDomIndex X Y (bouquetLeftElement X Y x).1.unop
2845   (bouquetLeftElement X Y x).2 = X.G.obj x := by
2846   rcases x with <|<|m|>, k|>
2847   simp only [bouquetLeftElement, bouquetLeftPred, bouquetRawDomIndex,
2848     bouquetChain, Fin.cases_succ]
2849   split
2850   . rename_i k' hk
2851     change Sum.inl _ = Sum.inl k' at hk
2852     have hk' := Sum.inl.inj hk
2853     subst k'
2854     congr 1
2855     apply Functor.Elements.ext (F := X.P.H) _ _
2856       (congrArg op (X.P.folio.paddedIndex_fin m))
2857     dsimp only [Pag.cell]
2858     change X.P.H.map (eqToHom _) (X.P.H.map (eqToHom _) k) = k
2859     rw [<- FunctorToTypes.map_comp_apply]
2860     simp
2861   . rename_i k' hk
2862     change Sum.inl _ = Sum.inr k' at hk
2863     cases hk
2864
2865 theorem bouquetRightRawDom_eq (X Y : Atl) (y : Y.E) :
2866   bouquetRawDomIndex X Y (bouquetRightElement X Y y).1.unop
2867   (bouquetRightElement X Y y).2 = Y.G.obj y := by
2868   rcases y with <|<|m|>, k|>
2869   simp only [bouquetRightElement, bouquetRightPred, bouquetRawDomIndex,
2870     bouquetChain, Fin.cases_succ]
2871   split
2872   . rename_i k' hk
2873     change Sum.inr _ = Sum.inl k' at hk
2874     cases hk
2875   . rename_i k' hk
2876     change Sum.inr _ = Sum.inr k' at hk
2877     have hk' := Sum.inr.inj hk
2878     subst k'
2879     congr 1
2880     apply Functor.Elements.ext (F := Y.P.H) _ _
2881       (congrArg op (Y.P.folio.paddedIndex_fin m))
2882     dsimp only [Pag.cell]
2883     change Y.P.H.map (eqToHom _) (Y.P.H.map (eqToHom _) k) = k
2884     rw [<- FunctorToTypes.map_comp_apply]
2885     simp
2886
2887 def paddedElement (X : Atl) (x : X.E) : X.E :=
2888   X.P.cell (op (X.P.folio.paddedIndex x.1.unop.1))
2889   (X.P.H.map
2890     (eqToHom (congrArg op (X.P.folio.paddedIndex_fin x.1.unop).symm)) x.2)
2891
2892 theorem paddedElement_eq (X : Atl) (x : X.E) : paddedElement X x = x := by
2893   apply Functor.Elements.ext (F := X.P.H) _ _
2894     (congrArg op (X.P.folio.paddedIndex_fin x.1.unop))
2895   dsimp only [paddedElement, Pag.cell]
2896   change X.P.H.map (eqToHom _) (X.P.H.map (eqToHom _) x.2) = x.2
2897   rw [<- FunctorToTypes.map_comp_apply]
2898   simp
2899
2900 def bouquetRootIndex (X Y : Atl) :
2901   forall (m : Fin (bouquetLength X.P.folio Y.P.folio))
2902     (v : (bouquetFolio X.P.folio Y.P.folio).H.obj (op m)),
2903     bouquetRawDomIndex X Y m v --> domSum (extent X) (extent Y) := by
2904   intro m
2905   induction m using Fin.cases with
2906   | zero => intro v; exact Id _
2907   | succ n =>
2908     intro v
2909     change (match ofLex v with
2910       | .inl k => X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
2911       | .inr k => Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k))
2912       --> domSum (extent X) (extent Y)
2913     exact match ofLex v with

```

```

2914 | .inl k => X.G.map (X.P.cellToOrigin
2915   (op (X.P.folio.paddedIndex n.1)) k) >>
2916   domSum.inl (extent X) (extent Y)
2917 | .inr k => Y.G.map (Y.P.cellToOrigin
2918   (op (Y.P.folio.paddedIndex n.1)) k) >>
2919   domSum.inr (extent X) (extent Y)
2920
2921 @[simp]
2922 theorem bouquetRootIndex_succ_left (X Y : At1)
2923   (n : Fin (bouquetDepth X.P.folio Y.P.folio))
2924   (k : X.P.H.obj (op (X.P.folio.paddedIndex n.1)))
2925   (a : X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)) :
2926   bouquetRootIndex X Y n.succ (toLex (Sum.inl k)) a =
2927     Sum.inl (X.G.map
2928       (X.P.cellToOrigin (op (X.P.folio.paddedIndex n.1)) k) a) := by
2929   rfl
2930
2931 @[simp]
2932 theorem bouquetRootIndex_succ_right (X Y : At1)
2933   (n : Fin (bouquetDepth X.P.folio Y.P.folio))
2934   (k : Y.P.H.obj (op (Y.P.folio.paddedIndex n.1)))
2935   (a : Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k)) :
2936   bouquetRootIndex X Y n.succ (toLex (Sum.inr k)) a =
2937     Sum.inr (Y.G.map
2938       (Y.P.cellToOrigin (op (Y.P.folio.paddedIndex n.1)) k) a) := by
2939   rfl
2940
2941 theorem bouquetRootIndex_left (X Y : At1) (x : X.E)
2942   (a : bouquetRawDomIndex X Y (bouquetLeftElement X Y x).1.unop
2943     (bouquetLeftElement X Y x).2) :
2944   bouquetRootIndex X Y (bouquetLeftElement X Y x).1.unop
2945     (bouquetLeftElement X Y x).2 a =
2946     Sum.inl (originImage X x (bouquetLeftRawDom_eq X Y x transport a)) := by
2947   rcases x with <|<m|>, k|>
2948   change bouquetRootIndex X Y (bouquetLeftPred X Y m).succ
2949     (toLex (Sum.inl (X.P.H.map
2950       (eqToHom (congrArg op (X.P.folio.paddedIndex_fin m).symm)) k))) a = _
2951   rw [bouquetRootIndex_succ_left]
2952   congr 1
2953   change X.G.map (X.P.folio.toOrigin (paddedElement X <|op m, k|>)) a =
2954     X.G.map (X.P.folio.toOrigin <|op m, k|>)
2955     (bouquetLeftRawDom_eq X Y <|op m, k|> transport a)
2956   let q := paddedElement_eq X (<|op m, k|> : X.E)
2957   have hto : eqToHom q >> X.P.folio.toOrigin <|op m, k|> =
2958     X.P.folio.toOrigin (paddedElement X <|op m, k|>) := by
2959     apply CategoryOfElements.ext
2960     apply Subsingleton.elim
2961   rw [← hto, X.G.map_comp]
2962   change X.G.map (X.P.folio.toOrigin <|op m, k|>)
2963     (X.G.map (eqToHom q) a) =
2964     X.G.map (X.P.folio.toOrigin <|op m, k|>)
2965     (bouquetLeftRawDom_eq X Y <|op m, k|> transport a)
2966   apply congrArg (fun z => X.G.map (X.P.folio.toOrigin <|op m, k|>) z)
2967   have hmap := eqToHom_map X.G q
2968   rw [hmap]
2969   have heq : congrArg X.G.obj q = bouquetLeftRawDom_eq X Y <|op m, k|> :=
2970     Subsingleton.elim _ _
2971   cases heq
2972   exact domIns_eqToHom_apply _ _
2973
2974 theorem bouquetRootIndex_right (X Y : At1) (y : Y.E)
2975   (a : bouquetRawDomIndex X Y (bouquetRightElement X Y y).1.unop
2976     (bouquetRightElement X Y y).2) :
2977   bouquetRootIndex X Y (bouquetRightElement X Y y).1.unop
2978     (bouquetRightElement X Y y).2 a =
2979     Sum.inr (originImage Y y (bouquetRightRawDom_eq X Y y transport a)) := by
2980   rcases y with <|<m|>, k|>
2981   change bouquetRootIndex X Y (bouquetRightPred X Y m).succ
2982     (toLex (Sum.inr (Y.P.H.map
2983       (eqToHom (congrArg op (Y.P.folio.paddedIndex_fin m).symm)) k))) a = _
2984   rw [bouquetRootIndex_succ_right]
2985   congr 1
2986   change Y.G.map (Y.P.folio.toOrigin (paddedElement Y <|op m, k|>)) a =

```



```

2987   Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
2988   (bouquetRightRawDom_eq X Y <|op m, k|> transport a)
2989   let q := paddedElement_eq Y (<|op m, k|> : Y.E)
2990   have hto : eqToHom q >> Y.P.folio.toOrigin <|op m, k|> =
2991     Y.P.folio.toOrigin (paddedElement Y <|op m, k|>) := by
2992     apply CategoryOfElements.ext
2993     apply Subsingleton.elim
2994   rw [← hto, Y.G.map_comp]
2995   change Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
2996     (Y.G.map (eqToHom q) a) =
2997     Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
2998     (bouquetRightRawDom_eq X Y <|op m, k|> transport a)
2999   apply congrArg (fun z => Y.G.map (Y.P.folio.toOrigin <|op m, k|>) z)
3000   have hmap := eqToHom_map Y.G q
3001   rw [hmap]
3002   have heq : congrArg Y.G.obj q = bouquetRightRawDom_eq X Y <|op m, k|> :=
3003     Subsingleton.elim _ _
3004   cases heq
3005   exact domIns_eqToHom_apply _ _
3006
3007   def bouquetDom (X Y : Atl) (x : (bouquetPag X Y).E) : DomIns :=
3008     imageDom (bouquetRootIndex X Y x.1.unop x.2)
3009
3010   theorem bouquetRange_mono (X Y : Atl) {x y : (bouquetPag X Y).E}
3011     (f : x --> y) :
3012     Set.range (bouquetRootIndex X Y x.1.unop x.2) subset
3013     Set.range (bouquetRootIndex X Y y.1.unop y.2) := by
3014     intro z hz
3015     rcases x with <|<|mx|>, vx|>
3016     rcases y with <|<|my|>, vy|>
3017     rcases hz with <|a, ha|>
3018     rw [← ha]
3019     induction mx using Fin.cases with
3020     | zero =>
3021       induction my using Fin.cases with
3022       | zero => exact <|a, rfl|>
3023       | succ p =>
3024         have hf : p.succ -->
3025           (<|0, bouquetLength_pos X.P.folio Y.P.folio|> :
3026             Fin (bouquetLength X.P.folio Y.P.folio)) :=
3027           Quiver.Hom.unop f.val
3028         have h := leOfHom hf
3029         change p.1 + 1 <= 0 at h
3030         omega
3031     | succ n =>
3032       induction my using Fin.cases with
3033       | zero => exact <|bouquetRootIndex X Y n.succ vx a, rfl|>
3034       | succ p =>
3035         have hf : p.succ --> n.succ := Quiver.Hom.unop f.val
3036         have hpn : p <= n := Fin.succ_le_succ_iff.mp (leOfHom hf)
3037         let hxp : X.P.folio.paddedIndex p.1 <= X.P.folio.paddedIndex n.1 :=
3038           X.P.folio.paddedIndex_mono hpn
3039         let hyp : Y.P.folio.paddedIndex p.1 <= Y.P.folio.paddedIndex n.1 :=
3040           Y.P.folio.paddedIndex_mono hpn
3041         generalize hvx : ofLex vx = s
3042         rcases s with k | k
3043         . have evx : vx = toLex (Sum.inl k) :=
3044           ofLex.injective (by simp using hvx)
3045         subst vx
3046         let k' := X.P.H.map (homOfLE hxp).op k
3047         have ef : f.val = (homOfLE (show p.succ <= n.succ from
3048           Fin.succ_le_succ_iff.mpr hpn)).op := Subsingleton.elim _ _
3049         have hfv : (bouquetPag X Y).H.map f.val (toLex (Sum.inl k)) = vy :=
3050           f.property
3051         have hvy : ofLex vy = Sum.inl k' := by
3052           rw [← hfv, ef]
3053           change ofLex (toLex (Sum.map _ _ (ofLex (toLex (Sum.inl k))))) = _
3054           simp [k']
3055           congr 2
3056         have evy : vy = toLex (Sum.inl k') :=
3057           ofLex.injective (by simp using hvy)
3058         subst vy
3059         let q := CategoryOfElements.homMk

```



```

3060      (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
3061      (X.P.cell (op (X.P.folio.paddedIndex p.1)) k')
3062      (homOfLE hxp).op rfl
3063      refine <|X.G.map q a, ?_|>
3064      change Sum.inl (X.G.map (X.P.cellToOrigin _ k') (X.G.map q a)) =
3065      Sum.inl (X.G.map (X.P.cellToOrigin _ k) a)
3066      apply congrArg Sum.inl
3067      have hc : q >> X.P.cellToOrigin _ k' = X.P.cellToOrigin _ k :=
3068      CategoryOfElements.ext X.P.H _ _ (Subsingleton.elim _ _)
3069      have hm := X.G.map_comp q (X.P.cellToOrigin _ k')
3070      rw [hc] at hm
3071      exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
3072      · have evx : vx = toLex (Sum.inr k) :=
3073      ofLex.injective (by simp using hvx)
3074      subst vx
3075      let k' := Y.P.H.map (homOfLE hyp).op k
3076      have ef : f.val = (homOfLE (show p.succ <= n.succ from
3077      Fin.succ_le_succ_iff.mpr hpn)).op := Subsingleton.elim _ _
3078      have hfv : (bouquetPag X Y).H.map f.val (toLex (Sum.inr k)) = vy :=
3079      f.property
3080      have hvy : ofLex vy = Sum.inr k' := by
3081      rw [<- hfv, ef]
3082      change ofLex (toLex (Sum.map _ _ (ofLex (toLex (Sum.inr k))))) = _
3083      simp [k']
3084      congr 2
3085      have evy : vy = toLex (Sum.inr k') :=
3086      ofLex.injective (by simp using hvy)
3087      subst vy
3088      let q := CategoryOfElements.homMk
3089      (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k)
3090      (Y.P.cell (op (Y.P.folio.paddedIndex p.1)) k')
3091      (homOfLE hyp).op rfl
3092      refine <|Y.G.map q a, ?_|>
3093      change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k') (Y.G.map q a)) =
3094      Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a)
3095      apply congrArg Sum.inr
3096      have hc : q >> Y.P.cellToOrigin _ k' = Y.P.cellToOrigin _ k :=
3097      CategoryOfElements.ext Y.P.H _ _ (Subsingleton.elim _ _)
3098      have hm := Y.G.map_comp q (Y.P.cellToOrigin _ k')
3099      rw [hc] at hm
3100      exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
3101
3102 @[simp]
3103 theorem imageDom.map_val {A B R : DomIns} (f : A --> R) (g : B --> R)
3104 (h : Set.range f subset Set.range g) (z : imageDom f) :
3105 (imageDom.map f g h z).1 = z.1 := rfl
3106
3107 def bouquetImageMap (X Y : Atl) {x y : (bouquetPag X Y).E} (f : x --> y) :
3108 bouquetDom X Y x --> bouquetDom X Y y :=
3109 imageDom.map _ _ (bouquetRange_mono X Y f)
3110
3111 def bouquetFunctor (X Y : Atl) : (bouquetPag X Y).E ==> DomIns where
3112 obj := bouquetDom X Y
3113 map := bouquetImageMap X Y
3114 map_id := by
3115   intro x
3116   apply DomIns.hom_ext
3117   intro z
3118   apply Subtype.ext
3119   change z.1 = z.1
3120   rfl
3121 map_comp := by
3122   intro x y z f g
3123   apply DomIns.hom_ext
3124   intro w
3125   apply Subtype.ext
3126   change w.1 = w.1
3127   rfl
3128
3129 def bouquetAtlas (X Y : Atl) : Atl where
3130 P := bouquetPag X Y
3131 G := bouquetFunctor X Y
3132 disjoint := by

```

```

3133 intro m i j hij x y hxy
3134 rcases m with <|m|>
3135 induction m using Fin.cases with
3136 | zero =>
3137   exact hij ((bouquetFolio X.P.folio Y.P.folio).originEquiv.injective
3138     (Subsingleton.elim _ _))
3139 | succ n =>
3140   rcases x.2 with <|a, ha|>
3141   rcases y.2 with <|b, hb|>
3142   have huv : x.1 = y.1 := by
3143     have e := congrArg (fun q => q.1) hxy
3144     change x.1 = y.1 at e
3145     exact e
3146   have hroot : bouquetRootIndex X Y n.succ i a =
3147     bouquetRootIndex X Y n.succ j b := ha.trans (huv.trans hb.symm)
3148   generalize hi : ofLex i = si
3149   generalize hj : ofLex j = sj
3150   rcases si with k | k <; rcases sj with l | l
3151   . have ei : i = toLex (Sum.inl k) :=
3152     ofLex.injective (by simpa using hi)
3153   have ej : j = toLex (Sum.inl l) :=
3154     ofLex.injective (by simpa using hj)
3155   subst i
3156   subst j
3157   have hkl : k != l := by
3158     intro e
3159     subst l
3160     exact hij rfl
3161   change Sum.inl (X.G.map (X.P.cellToOrigin _ k) a) =
3162     Sum.inl (X.G.map (X.P.cellToOrigin _ l) b) at hroot
3163   exact X.disjoint _ k l hkl a b (Sum.inl.inj hroot)
3164   . have ei : i = toLex (Sum.inl k) :=
3165     ofLex.injective (by simpa using hi)
3166   have ej : j = toLex (Sum.inr l) :=
3167     ofLex.injective (by simpa using hj)
3168   subst i
3169   subst j
3170   change Sum.inl (X.G.map (X.P.cellToOrigin _ k) a) =
3171     Sum.inr (Y.G.map (Y.P.cellToOrigin _ l) b) at hroot
3172   exact Sum.noConfusion hroot
3173   . have ei : i = toLex (Sum.inr k) :=
3174     ofLex.injective (by simpa using hi)
3175   have ej : j = toLex (Sum.inl l) :=
3176     ofLex.injective (by simpa using hj)
3177   subst i
3178   subst j
3179   change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a) =
3180     Sum.inl (X.G.map (X.P.cellToOrigin _ l) b) at hroot
3181   exact Sum.noConfusion hroot
3182   . have ei : i = toLex (Sum.inr k) :=
3183     ofLex.injective (by simpa using hi)
3184   have ej : j = toLex (Sum.inr l) :=
3185     ofLex.injective (by simpa using hj)
3186   subst i
3187   subst j
3188   have hkl : k != l := by
3189     intro e
3190     subst l
3191     exact hij rfl
3192   change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a) =
3193     Sum.inr (Y.G.map (Y.P.cellToOrigin _ l) b) at hroot
3194   exact Y.disjoint _ k l hkl a b (Sum.inr.inj hroot)
3195
3196 /-- The object-level atlas merge used by the horizontal construction. -/
3197 def AtlMerge (X Y : Atl) : Atl := bouquetAtlas X Y
3198
3199 def bouquetTallPred (X Y : Atl) (n : Nat) :
3200   Fin (bouquetDepth X.P.folio Y.P.folio) :=
3201   <|min n (bouquetDepth X.P.folio Y.P.folio - 1), by
3202     have hp : 0 < bouquetDepth X.P.folio Y.P.folio :=
3203       lt_of_lt_of_le X.P.folio.positive (le_max_left _ _)
3204     omega|>
3205

```

```

3206 theorem bouquet_padded_succ (X Y : Atl) (n : Nat) :
3207   (bouquetFolio X.P.folio Y.P.folio).paddedIndex (n + 1) =
3208   (bouquetTallPred X Y n).succ := by
3209   apply Fin.ext
3210   simp only [Folio.paddedIndex, bouquetFolio, bouquetLength, bouquetTallPred,
3211     bouquetDepth, Fin.val_succ]
3212   have hp : 0 < max X.P.folio.length Y.P.folio.length :=
3213     lt_of_lt_of_le X.P.folio.positive (le_max_left _ _)
3214   omega
3215
3216 theorem left_padded_tallPred (X Y : Atl) (n : Nat) :
3217   X.P.folio.paddedIndex (bouquetTallPred X Y n).1 =
3218   X.P.folio.paddedIndex n := by
3219   apply Fin.ext
3220   simp only [Folio.paddedIndex, bouquetTallPred, bouquetDepth]
3221   have hx : X.P.folio.length <= max X.P.folio.length Y.P.folio.length :=
3222     le_max_left _ _
3223   omega
3224
3225 theorem right_padded_tallPred (X Y : Atl) (n : Nat) :
3226   Y.P.folio.paddedIndex (bouquetTallPred X Y n).1 =
3227   Y.P.folio.paddedIndex n := by
3228   apply Fin.ext
3229   simp only [Folio.paddedIndex, bouquetTallPred, bouquetDepth]
3230   have hy : Y.P.folio.length <= max X.P.folio.length Y.P.folio.length :=
3231     le_max_right _ _
3232   omega
3233
3234 /-- The finite coherent implementation of `AtlMerge` presents exactly the
3235 componentwise positive pages used by `tallMerge`. -/
3236 def AtlMerge.pageEquiv (X Y : Atl) (n : Nat) :
3237   (AtlMerge X Y).page (n + 1) Equiv Sum (X.page n) (Y.page n) := by
3238   change (bouquetChain X.P.folio Y.P.folio
3239     ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (n + 1))).Obj Equiv _
3240   rw [bouquet_padded_succ]
3241   change Lex (Sum
3242     ((X.P.folio.core.obj (X.P.folio.paddedIndex
3243       (bouquetTallPred X Y n).1)).unop.Obj)
3244     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
3245       (bouquetTallPred X Y n).1)).unop.Obj)) Equiv _
3246   rw [left_padded_tallPred, right_padded_tallPred]
3247   exact ofLex
3248
3249 /-- The left input as a page-preserving subobject of the tall presentation of
3250 an atlas merge. In particular, occurrence `n` is sent to occurrence `n+1`;
3251 no finite representative is selected here. -/
3252 def tallBouquetLeftElement (X Y : Atl) (x : X.tall.E) : (AtlMerge X Y).tall.E := by
3253   refine <|op (x.1.unop + 1), ?_|>
3254   change (bouquetChain X.P.folio Y.P.folio
3255     ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (x.1.unop + 1))).Obj
3256   rw [bouquet_padded_succ]
3257   change Lex (Sum
3258     ((X.P.folio.core.obj (X.P.folio.paddedIndex
3259       (bouquetTallPred X Y x.1.unop).1)).unop.Obj)
3260     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
3261       (bouquetTallPred X Y x.1.unop).1)).unop.Obj))
3262   rw [left_padded_tallPred]
3263   exact toLex (Sum.inl x.2)
3264
3265 /-- The right input in the tall presentation of an atlas merge. -/
3266 def tallBouquetRightElement (X Y : Atl) (y : Y.tall.E) : (AtlMerge X Y).tall.E := by
3267   refine <|op (y.1.unop + 1), ?_|>
3268   change (bouquetChain X.P.folio Y.P.folio
3269     ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (y.1.unop + 1))).Obj
3270   rw [bouquet_padded_succ]
3271   change Lex (Sum
3272     ((X.P.folio.core.obj (X.P.folio.paddedIndex
3273       (bouquetTallPred X Y y.1.unop).1)).unop.Obj)
3274     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
3275       (bouquetTallPred X Y y.1.unop).1)).unop.Obj))
3276   rw [right_padded_tallPred]
3277   exact toLex (Sum.inr y.2)
3278

```

```

3279 /-- The category of Atlas Federations used for iterated stable atlas merging. -/
3280 abbrev AtlFed := StableAtlasFamily
3281
3282 /-- The horizontal bifunctor is federation merge on objects and applies a
3283 stable traversal independently to every tagged component on arrows. -/
3284 def AtlHorSum : AtlFed * AtlFed ==> AtlFed := StableAtlHorSum
3285
3286 def AtlBrd (X Y : AtlFed) :
3287   AtlHorSum.obj (X, Y) Iso AtlHorSum.obj (Y, X) :=
3288   stableAtlasBraiding X Y
3289
3290 def AtlAsoc (X Y Z : AtlFed) :
3291   AtlHorSum.obj (AtlHorSum.obj (X, Y), Z) Iso
3292   AtlHorSum.obj (X, AtlHorSum.obj (Y, Z)) :=
3293   stableAtlasAssociator X Y Z
3294
3295 def AtlLu (X : AtlFed) : AtlHorSum.obj (stableAtlasUnit, X) Iso X :=
3296   stableAtlasLeftUnitor X
3297
3298 def AtlRu (X : AtlFed) : AtlHorSum.obj (X, stableAtlasUnit) Iso X :=
3299   stableAtlasRightUnitor X
3300
3301 theorem horizontalLemma : Nonempty (SymmetricCategory AtlFed) :=
3302   <|inferInstance|>
3303
3304 instance : SymmetricCategory StableAtlasFamilyop where
3305   symmetry X Y := by
3306     apply Quiver.Hom.unop_inj
3307     simp
3308
3309 theorem staDaTravInc_essImage (F : StaDaTravPresheaf) :
3310   StaDaTravInc.essImage F :=
3311   <|<|F, trivial|>, <|Iso.refl _|>|>
3312
3313 instance staDaTravPreservesTensorRightForTensor (v : Type 3)
3314   (d : StableAtlasFamilyop) :
3315   PreservesColimitsOfShape
3316   (CostructuredArrow (MonoidalCategory.tensor StableAtlasFamilyop) d)
3317   (MonoidalCategory.tensorRight v) :=
3318   preservesColimitsOfShape_of_natIso
3319   (BraidedCategory.tensorLeftIsoTensorRight v)
3320
3321 instance staDaTravPreservesTensorRightForUnit (v : Type 3)
3322   (d : StableAtlasFamilyop) :
3323   PreservesColimitsOfShape
3324   (CostructuredArrow
3325     (Functor.fromPUnit
3326       (MonoidalCategory.tensorUnit StableAtlasFamilyop)) d)
3327   (MonoidalCategory.tensorRight v) :=
3328   preservesColimitsOfShape_of_natIso
3329   (BraidedCategory.tensorLeftIsoTensorRight v)
3330
3331 instance staDaTravPreservesTensorRightForUnitProduct (v : Type 3)
3332   (d : StableAtlasFamilyop * StableAtlasFamilyop) :
3333   PreservesColimitsOfShape
3334   (CostructuredArrow
3335     ((Functor.id StableAtlasFamilyop).prod
3336       (Functor.fromPUnit
3337         (MonoidalCategory.tensorUnit StableAtlasFamilyop)))) d)
3338   (MonoidalCategory.tensorRight v) :=
3339   preservesColimitsOfShape_of_natIso
3340   (BraidedCategory.tensorLeftIsoTensorRight v)
3341
3342 instance staDaTravPreservesTensorRightForTensorProduct (v : Type 3)
3343   (d : StableAtlasFamilyop * StableAtlasFamilyop) :
3344   PreservesColimitsOfShape
3345   (CostructuredArrow
3346     ((MonoidalCategory.tensor StableAtlasFamilyop).prod
3347       (Functor.id StableAtlasFamilyop)) d)
3348   (MonoidalCategory.tensorRight v) :=
3349   preservesColimitsOfShape_of_natIso
3350   (BraidedCategory.tensorLeftIsoTensorRight v)
3351

```

```

3352 noncomputable def staDaTravMonoidal : MonoidalCategory StaDaTrav :=
3353   MonoidalCategory.monoidalOfHasDayConvolutions StaDaTravInc
3354   (ObjectProperty.fullyFaithfuliota IsStableDataTransversal)
3355   (fun _ _ => staDaTravInc_essImage _)
3356   (staDaTravInc_essImage _)
3357
3358 noncomputable instance : MonoidalCategory StaDaTrav := staDaTravMonoidal
3359
3360 noncomputable instance staDaTravLawful :
3361   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct
3362   StableAtlasFamilyop (Type 3) StaDaTrav :=
3363   MonoidalCategory.lawfulDayConvolutionMonoidalCategoryStructOfHasDayConvolutions
3364   StaDaTravInc (ObjectProperty.fullyFaithfuliota IsStableDataTransversal)
3365   (fun _ _ => staDaTravInc_essImage _)
3366   (staDaTravInc_essImage _)
3367
3368 noncomputable instance staDaTravDayConvolution (F G : StaDaTrav) :
3369   MonoidalCategory.DayConvolution (StaDaTravInc.obj F) (StaDaTravInc.obj G) :=
3370   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution
3371   StableAtlasFamilyop (Type 3) StaDaTrav F G
3372
3373 noncomputable instance staDaTravDayConvolutionRightNested (F G H : StaDaTrav) :
3374   MonoidalCategory.DayConvolution (StaDaTravInc.obj F)
3375   (MonoidalCategory.DayConvolution.convolution
3376   (StaDaTravInc.obj G) (StaDaTravInc.obj H)) :=
3377   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution_2
3378   StableAtlasFamilyop (Type 3) StaDaTrav F G H
3379
3380 noncomputable instance staDaTravDayConvolutionLeftNested (F G H : StaDaTrav) :
3381   MonoidalCategory.DayConvolution
3382   (MonoidalCategory.DayConvolution.convolution
3383   (StaDaTravInc.obj F) (StaDaTravInc.obj G))
3384   (StaDaTravInc.obj H) :=
3385   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution_2'
3386   StableAtlasFamilyop (Type 3) StaDaTrav F G H
3387
3388 noncomputable def staDaTravBraiding (F G : StaDaTrav) :
3389   MonoidalCategory.tensorObj F G Iso MonoidalCategory.tensorObj G F := by
3390   exact (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).preimageIso
3391   (MonoidalCategory.DayConvolution.braiding
3392   (StaDaTravInc.obj F) (StaDaTravInc.obj G))
3393
3394 theorem staDaTravInc_map_braiding_hom (F G : StaDaTrav) :
3395   StaDaTravInc.map (staDaTravBraiding F G).hom =
3396   (MonoidalCategory.DayConvolution.braiding
3397   (StaDaTravInc.obj F) (StaDaTravInc.obj G)).hom := by
3398   exact (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_preimage _
3399
3400 theorem staDaTravInc_map_tensorHom {F_1 F_2 G_1 G_2 : StaDaTrav}
3401   (f : F_1 --> F_2) (g : G_1 --> G_2) :
3402   StaDaTravInc.map (MonoidalCategory.tensorHom f g) =
3403   MonoidalCategory.DayConvolution.map
3404   (StaDaTravInc.map f) (StaDaTravInc.map g) := by
3405   simpa [StaDaTravInc, staDaTravLawful] using
3406   (MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.iota_map_tensorHom_hom_eq_tensorHom
3407   StableAtlasFamilyop (Type 3) StaDaTrav f g)
3408
3409 theorem staDaTravInc_map_associator_hom (F G H : StaDaTrav) :
3410   StaDaTravInc.map (MonoidalCategory.associator F G H).hom =
3411   (MonoidalCategory.DayConvolution.associator
3412   (StaDaTravInc.obj F) (StaDaTravInc.obj G) (StaDaTravInc.obj H)).hom := by
3413   simpa [StaDaTravInc, staDaTravLawful] using
3414   (MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.
3415   iota_map_associator_hom_eq_associator_hom
3416   StableAtlasFamilyop (Type 3) StaDaTrav F G H)
3417
3417 noncomputable instance staDaTravBraided : BraidedCategory StaDaTrav where
3418   braiding := staDaTravBraiding
3419   braiding_naturality_right := fun X { _ } f => by
3420     rw [← MonoidalCategory.id_tensorHom, ← MonoidalCategory.tensorHom_id]
3421     apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
3422     simp only [Functor.map_comp, staDaTravInc_map_tensorHom,
3423     staDaTravInc_map_braiding_hom]

```

```

3424     exact MonoidalCategory.DayConvolution.braiding_naturality_right
3425       (StaDaTravInc.obj X) (StaDaTravInc.map f)
3426   braiding_naturality_left := fun { _ _ } f Z => by
3427     rw [← MonoidalCategory.tensorHom_id, ← MonoidalCategory.id_tensorHom]
3428     apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
3429     simp only [Functor.map_comp, staDaTravInc_map_tensorHom,
3430       staDaTravInc_map_braiding_hom]
3431     exact MonoidalCategory.DayConvolution.braiding_naturality_left
3432       (StaDaTravInc.map f) (StaDaTravInc.obj Z)
3433   hexagon_forward := fun X Y Z => by
3434     apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
3435     simp only [Functor.map_comp, staDaTravInc_map_associator_hom,
3436       staDaTravInc_map_braiding_hom]
3437     rw [← MonoidalCategory.tensorHom_id, ← MonoidalCategory.id_tensorHom]
3438     simp only [staDaTravInc_map_tensorHom]
3439     exact MonoidalCategory.DayConvolution.hexagon_forward
3440       (StaDaTravInc.obj X) (StaDaTravInc.obj Y) (StaDaTravInc.obj Z)
3441   hexagon_reverse := fun X Y Z => by
3442     apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
3443     simp only [Functor.map_comp, staDaTravInc_map_braiding_hom]
3444     rw [← MonoidalCategory.id_tensorHom, ← MonoidalCategory.tensorHom_id]
3445     simp only [staDaTravInc_map_tensorHom]
3446     exact MonoidalCategory.DayConvolution.hexagon_reverse
3447       (StaDaTravInc.obj X) (StaDaTravInc.obj Y) (StaDaTravInc.obj Z)
3448
3449   set_option maxHeartbeats 2400000 in
3450   noncomputable instance : SymmetricCategory StaDaTrav where
3451     symmetry F G := by
3452       apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
3453       simp only [Functor.map_comp]
3454       exact MonoidalCategory.DayConvolution.symmetry
3455         (StaDaTravInc.obj F) (StaDaTravInc.obj G)
3456
3457   /-- `StaDaTravMon` is definitionally the category of stable data transversals
3458   equipped with the Day convolution instance above. -/
3459   abbrev StaDaTravMon := StaDaTrav
3460
3461   noncomputable def I : StaDaTravMon := MonoidalCategory.tensorUnit StaDaTravMon
3462
3463   noncomputable def I_dayConvolutionUnit :
3464     MonoidalCategory.DayConvolutionUnit (StaDaTravInc.obj I) :=
3465     MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolutionUnit
3466     StableAtlasFamilyop (Type 3) StaDaTravMon
3467
3468   noncomputable def HorSum : StaDaTravMon * StaDaTravMon ==> StaDaTravMon :=
3469     MonoidalCategory.tensor StaDaTravMon
3470
3471   noncomputable def Brd (F G : StaDaTravMon) :
3472     MonoidalCategory.tensorObj F G Iso MonoidalCategory.tensorObj G F :=
3473     staDaTravBraiding F G
3474
3475   noncomputable def Asoc (F G H : StaDaTravMon) :
3476     MonoidalCategory.tensorObj (MonoidalCategory.tensorObj F G) H Iso
3477     MonoidalCategory.tensorObj F (MonoidalCategory.tensorObj G H) :=
3478     MonoidalCategory.associator F G H
3479
3480   noncomputable def Lu (F : StaDaTravMon) :
3481     MonoidalCategory.tensorObj I F Iso F := MonoidalCategory.leftUnitor F
3482
3483   noncomputable def Ru (F : StaDaTravMon) :
3484     MonoidalCategory.tensorObj F I Iso F := MonoidalCategory.rightUnitor F
3485
3486   theorem serenityLemma : Nonempty (SymmetricCategory StaDaTravMon) :=
3487     <|inferInstance|>
3488   end
3489
3490   end Datra

```

Listing 1: Complete Lean formalization of DaTra